

群组动画 (Crowd Animation) 上

毛天露

Email: ltm@ict.ac.cn



©Francis Pérez



 alamy stock photo

HE007C
www.alamy.com





自然界中的群组——椋鸟的Murmuration现象

这一幕让人联想起
庄子《逍遥游》中的语句：
鲲之大，不知其几千里也；
化而为鸟，其名为鹏。



椋(liáng)鸟

群组的定义

- 在同一物理环境下拥有相同目的的一群个体，他们的行为有别于作为单独个体时的行为

(A large group of individuals in the same physical environment sharing a common goal and may act in a different way than when they are alone.)

(Benesh, 86 and Roloff, 81)

- 行为的三个层次：

感知、决策、执行（路径规划、运动）

- 运动的三个方面：

- 个体自身的运动倾向（约束）
整体位移（运动学）、肢体动作
- 个体与环境之间的交互
- 个体与个体之间的交互

涌现

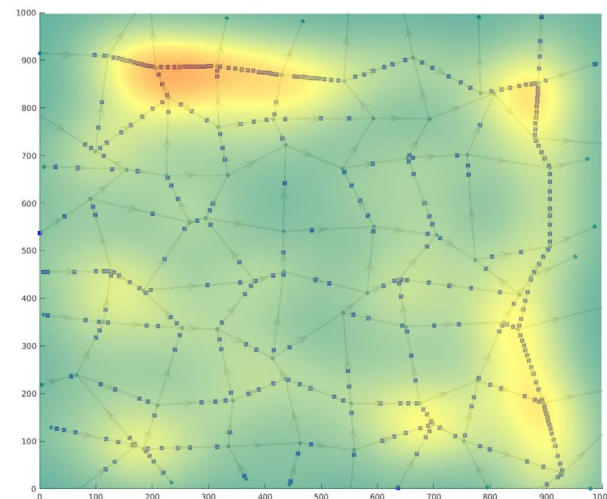
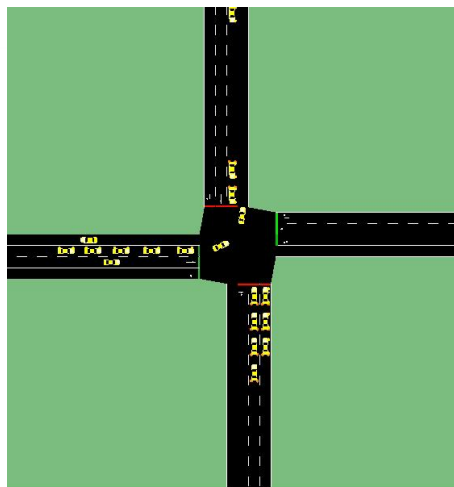


经典的群组动画方法分类

$$X_{i+1} = f(X_i, Environment, \dots)$$

微观方法

宏观方法



经典的群组动画方法分类

$$X_{i+1} = f(X_i, Environment, \dots)$$

规则驱动

微观方法

宏观方法

数据驱动

微观方法

宏观方法

联合

规则驱动的群组动画方法

- Boids / Steering模型
- 社会力模型
- RVO模型
- 连续群体模型

Boids模型



- 参考文献

Craig W. Reynolds. 1987. Flocks, herds and schools: A distributed behavioral model. *SIGGRAPH Comput. Graph.* 21, 4 (August 1987), 25-34.

Google citations: ~~14116次, 2023.10.15~~ **16000+, 2026.4**

如何模拟群体行为？

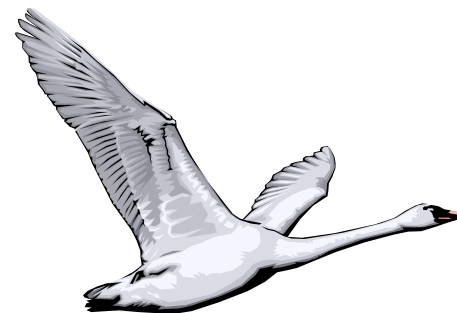
- 用脚本语言：难以描述！
- 粒子系统：太简单。
- 把粒子系统变得更加复杂
 - 引入物理仿真（飞行动力学）
 - 引入局部感知(Local perception)
 - 呈现群体行为
- 首先模拟一只鸟相对简单的行为，然后通过鸟个体之间大量的互动来呈现群体行为



引入物理仿真（飞行动力学）

飞行：沿着一条路径进行动态、增量和刚性运动

- 物体沿着一条三维曲线运动，并同时进行几何变换
- 物体是刚性运动，但是底层的几何模型在飞行坐标系中可随意改变形状（如鸟拍翅膀）
- 动量守恒
- 不超过最大速度、可指定最小速度
- **最大加速度**用来提供平稳的速度变化和航向



引入物理仿真（飞行动力学）

倾斜行为：在转弯时，为了对抗离心力而进行的侧倾，平衡受力，使得飞行更加平稳和自然。

- 仅考虑重力和离心力的相互作用，忽略翼面升力
- 根据当前速度和路径曲率计算转向加速度，曲率越大（即转弯越急），所需的转向加速度越大。
- 根据转向加速度和重力，计算物体需要倾斜的角度。
- 将倾斜角度应用到物体的局部坐标系，使其在视觉上表现出正确的倾斜效果。

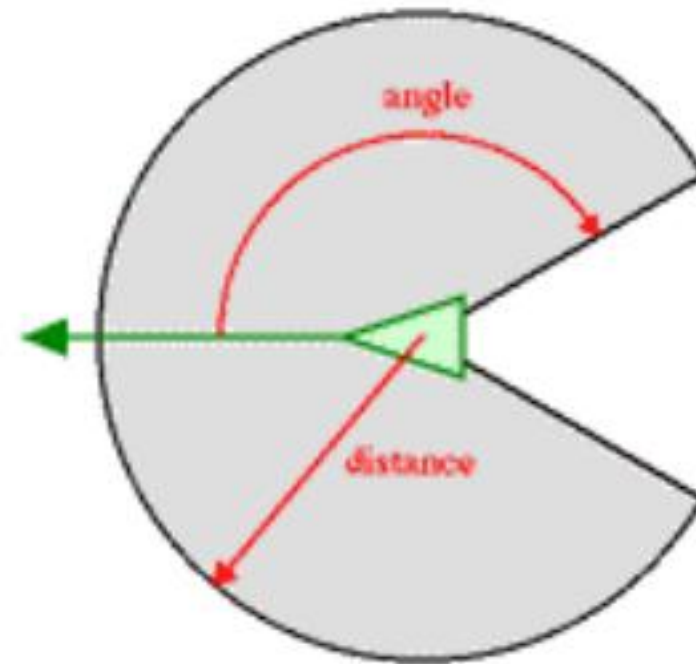


引入局部感知(Local perception)

- 鸟群中的鸟儿必须允许它与同伴进行运动协调
- 一只鸟不会把注意力放在鸟群中的所有鸟上
 - 鸟对鸟群中其它鸟的感知通常是局部和过滤过的
 - 它自己、它周围的邻居
- 两个平衡，但相反的行为
 - 与鸟群靠近的欲望
 - 与其它鸟不碰撞

引入局部感知(Local perception)

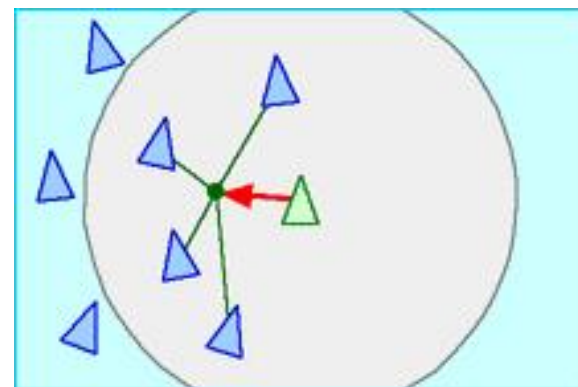
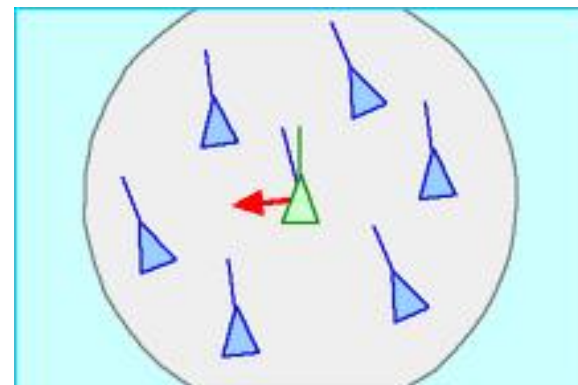
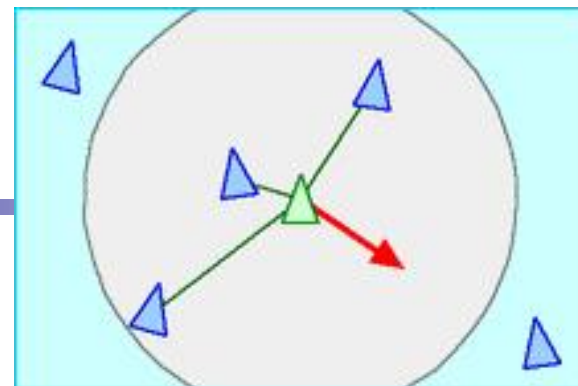
- 感知模型试图模拟真实的鸟
- 每个boid可以获得整个场景的几何描述，但是它只对周围的小部分邻居群组成员进行响应；
- 邻居群组成员由一个距离 (从boid的中心开始度量)和一角度(从boid的飞行方向开始度量)来刻画；
- 不在该范围的群体成员将被忽略，在该区域内的群体成员将影响boid的决策；



呈现群体行为

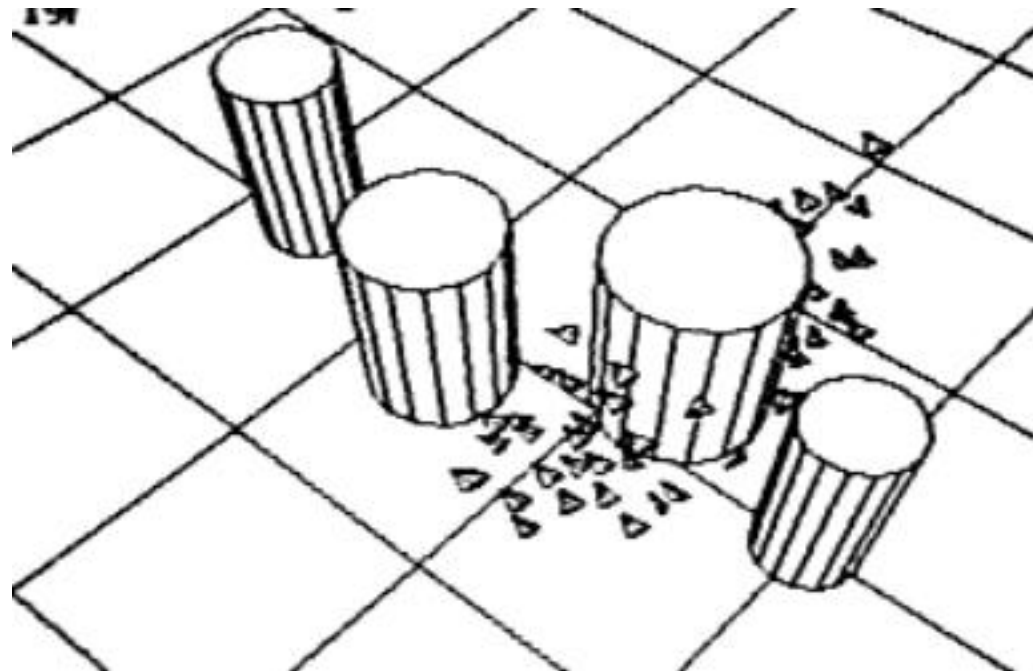
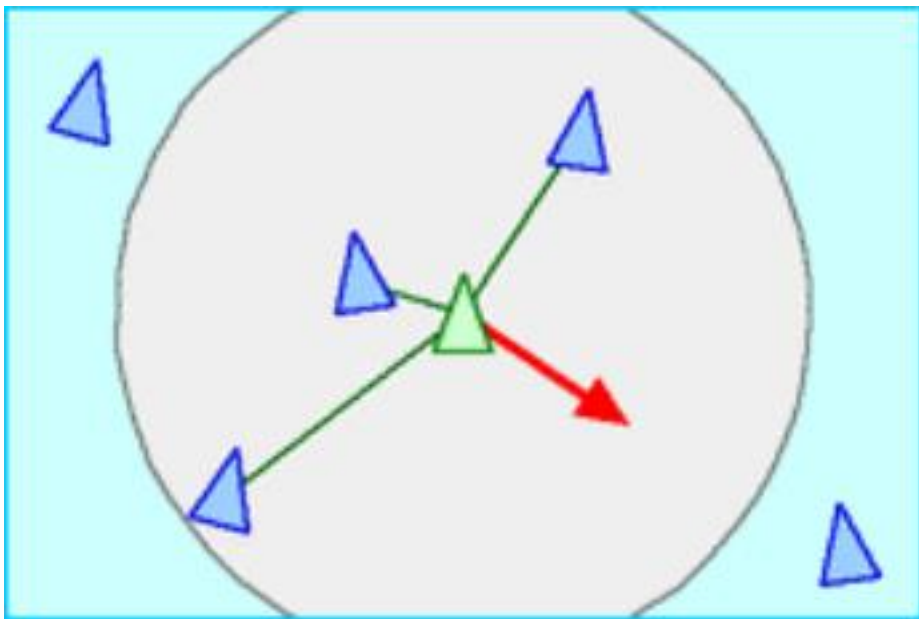
优先级递减的群体模拟三大原则

- 碰撞避免原则(Collision Avoidance)
 - 避免与相邻的群体成员相碰
- 速度匹配原则(Velocity Matching)
 - 尽量保持与周围邻居群体成员的速度匹配
- 群体合群原则(Flock Centering)
 - 群体成员尽量靠近



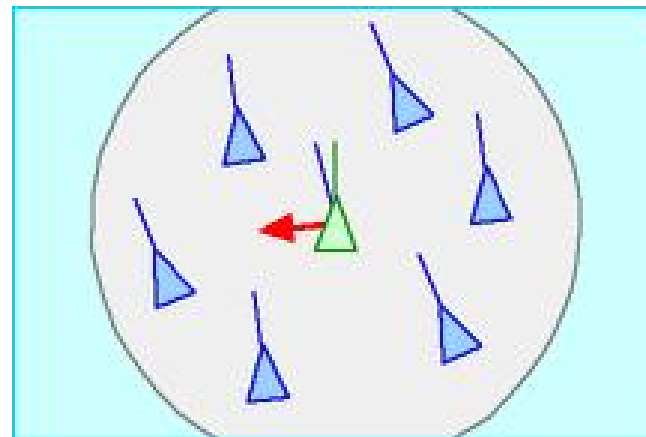
碰撞避免

- 努力摆脱迫在眉睫的影响
- 基于群体成员相对位置的静态碰撞避免
- 忽略速度



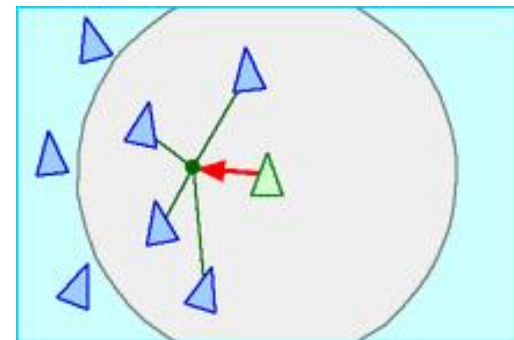
速度匹配原则(Velocity Matching)

- 只考虑速度
 - 忽视位置
- 碰撞避免预测版
 - 如果boid与相邻群体成员的速度匹配得很好，则将来不大可能发生碰撞
- 静态碰撞避免和动态速度匹配是相辅相成的
- 静态碰撞避免建立所需的最小分离距离，而速度匹配则尽量保持这个距离



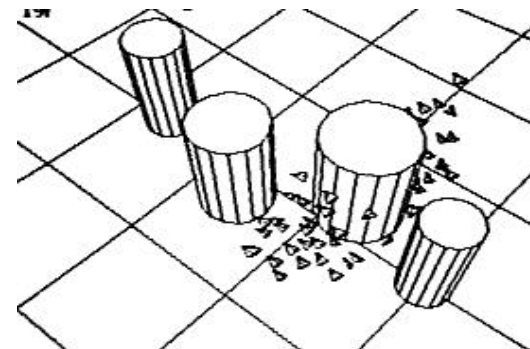
群体合群原则(Flock Centering)

- Boid局部感知的群体中心，实际上是相邻群体成员的中心；
- 使得boid飞向与相邻boids的中心；
- 如果一个boid接近群体的中心，该原则对它的影响不大（因为boid的密度将变均匀）；但如果它处于群体的边缘，这将对它有很大的影响；
- 允许模拟群体分裂和合并



仲裁独立的行为

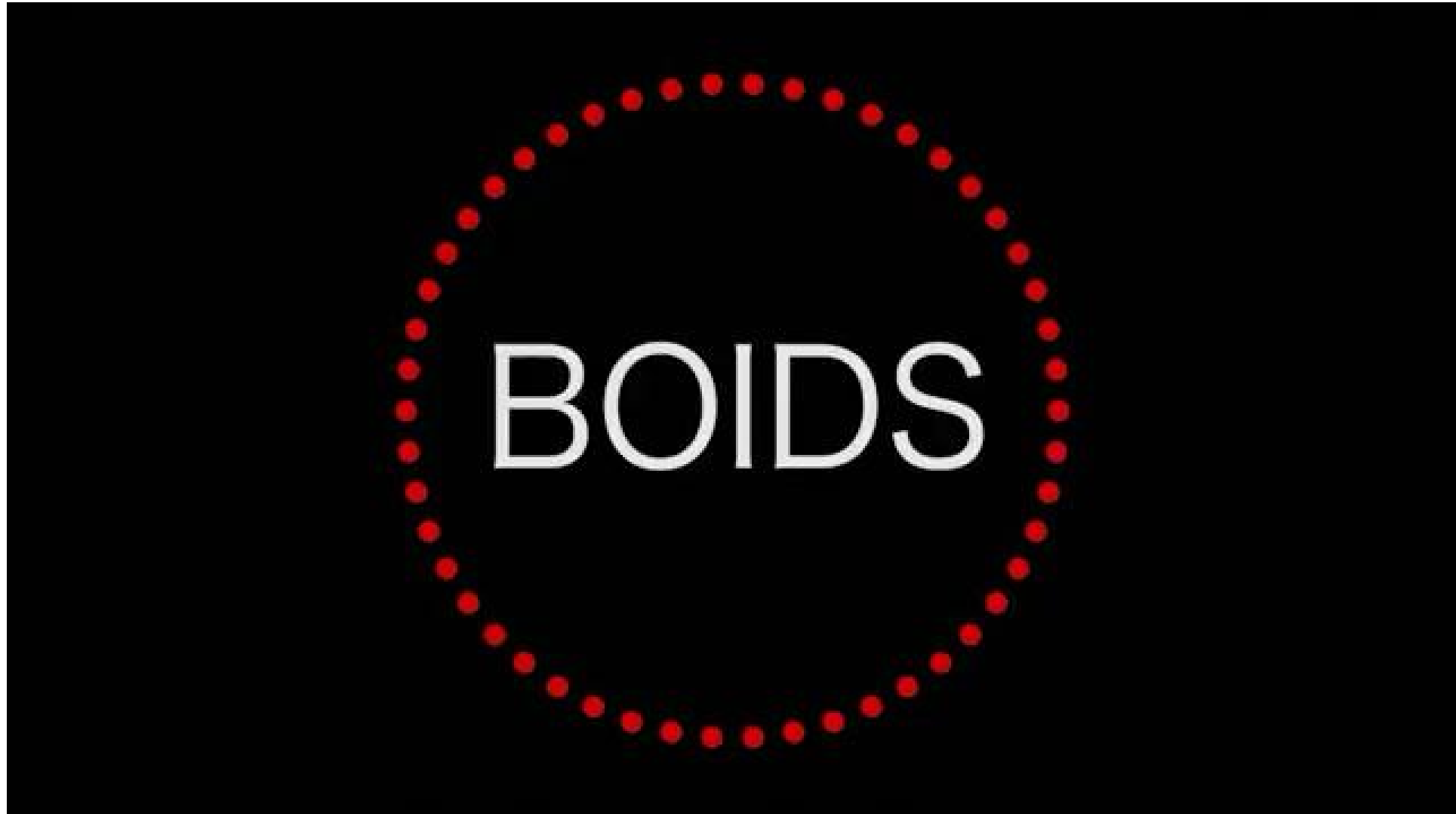
- 这三个行为表现为**广义的加速**请求
- 行为有多个参数，包括强度，进一步对加速请求进行调配
- boid大脑收集相关的加速请求，然后确定单一行为所需的加速
- 根据优先级进行加权平均
- 因为加权平均会取消相反的方向，在处理与障碍物的碰撞时，效果不好
- 根据优先级次序，把加速请求加入到累加器
- 强度添加到另一个累加器
- 继续执行，直到累加的强度的和大于最大加速度值
- 紧急加速先进行分配，以满足迫切需要
 - 例如: 忽略合群原则以回旋绕过障碍物



Flocking的整体控制

- 整体目标，全局方向
- 整体位置——鸟朝着这一个目标点飞行
- 有界加速，使得boid逐步飞向迁徙目标
- 对整体位置矢量、全局方向矢量进行动态设置
- 使得目标点沿着给定的路径变化

Boids模型的演示



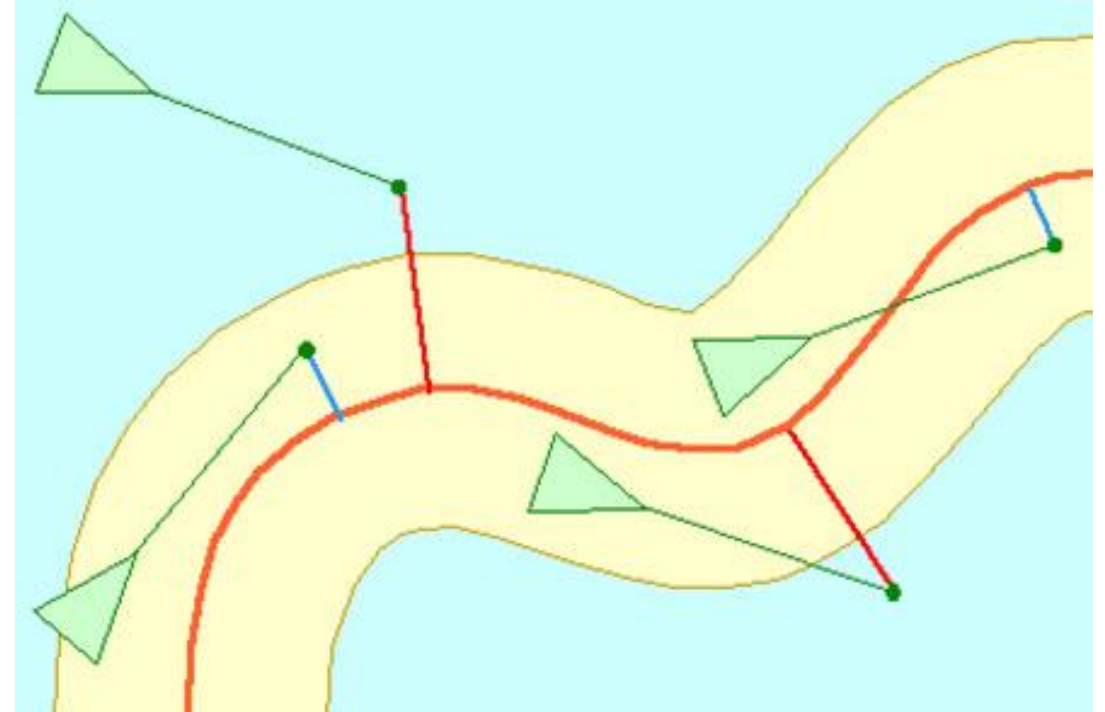
Steering Behaviors for Autonomous Characters

总结了自治角色的大部分基础行为

- 很多游戏引擎采用了这些技术：
如UNITY 3D, Cocos2d-x等。

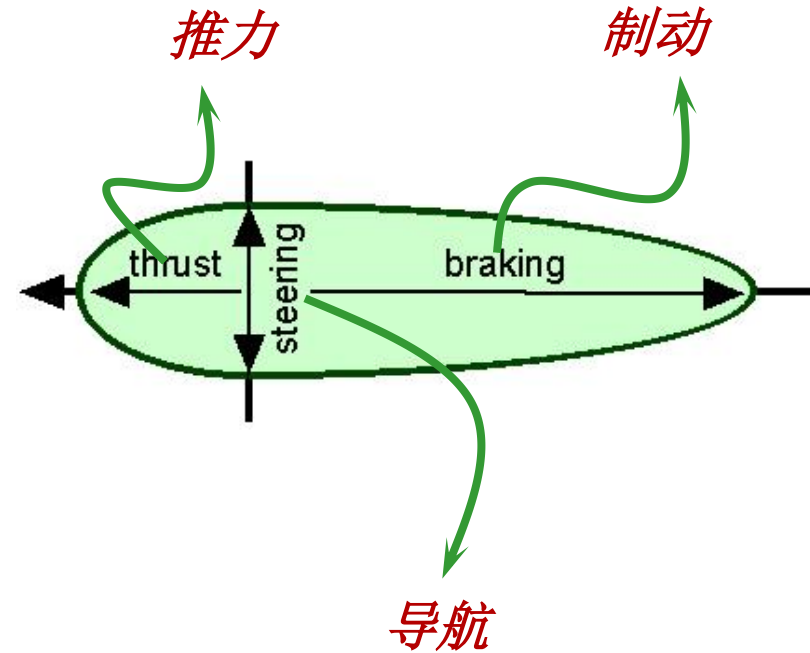
- 参考文献**

Craig W. Reynolds, Steering Behaviors For Autonomous Characters, *Game Developers Conference 1999*. (Google citations: **2271**, 2026.4)



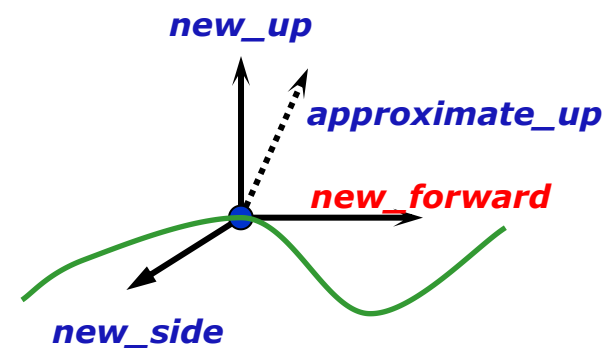
简单的车辆模型(Simple Vehicle Model)

- Simple Vehicle Model
 - mass scalar
 - position vector
 - velocity vector
 - max_force scalar
 - max_speed scalar
 - Orientation N basis vectors



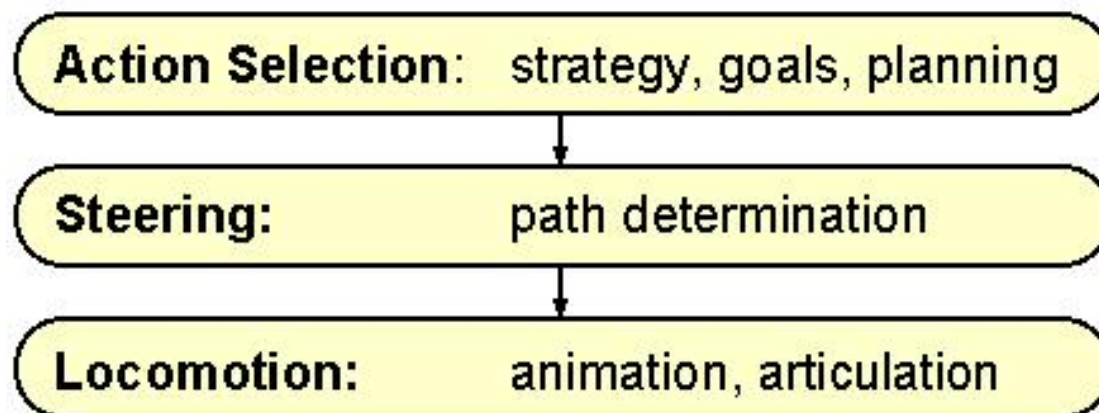
简单的车辆模型(Simple Vehicle Model)

- 简单车辆模型的物理原理为前向欧拉积分法：
 - **steering_force** = truncate (steering_direction, max_force)
acceleration = steering_force / mass
velocity = truncate (velocity + acceleration, max_speed)
position = position + velocity
- 构造新的基向量 (局部坐标系) :
 - **new_forward** = normalize (velocity)
 - approximate_up** = normalize (approximate_up) // if needed
 - new_side** = cross (new_forward, approximate_up)
 - new_up** = cross (new_side, new_forward)

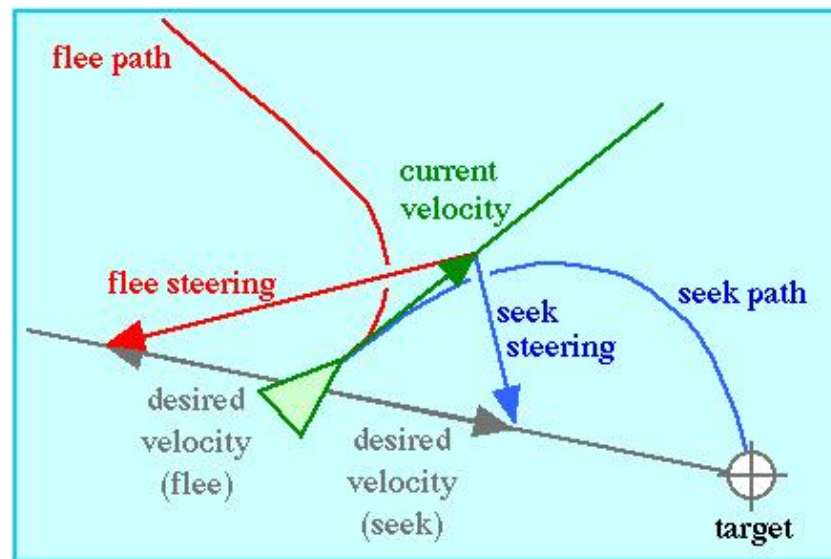


从行为到运动

- 自治角色运动行为的层次结构

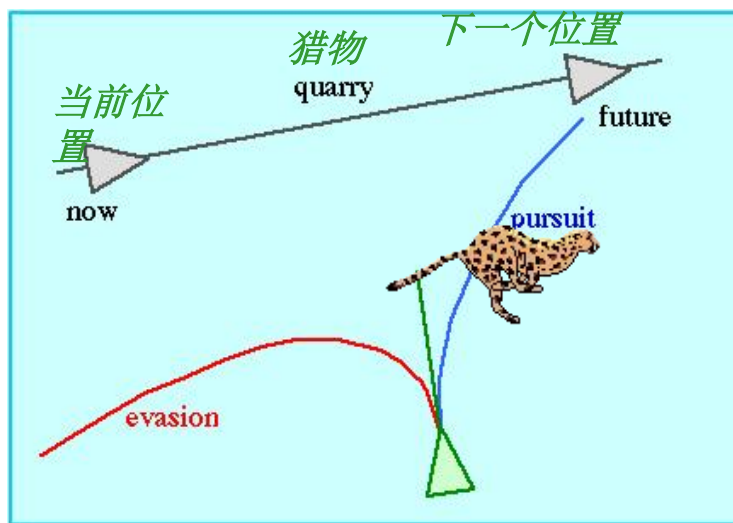


寻找和逃离(Seek and Flee)



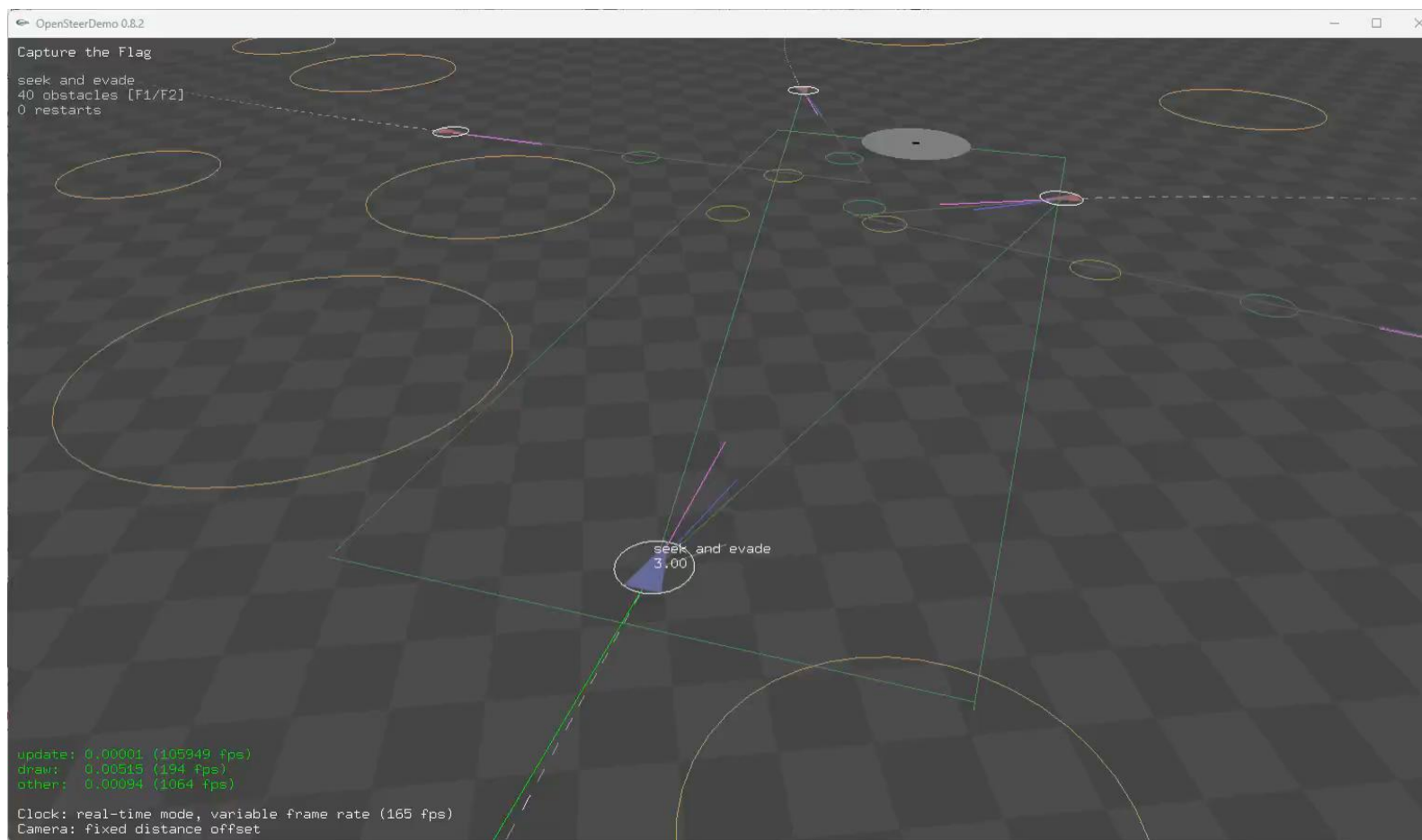
- 寻找(或追逐)一个静态的目标
 - **desired_velocity** = $\text{normalize}(\text{target} - \text{position}) * \text{max_speed}$
steering = **desired_velocity** - velocity
 - 把追逐反过来, 即为逃离
- <http://www.red3d.com/cwr/steer/SeekFlee.html>

追逐和躲避(Pursuit and Evasion)



- 追逐(Pursuit)与寻找(Seek)非常类似，其区别在于目标为一移动的角色(猎物)。
 - 假设猎物在预测区间 T 内不会转向。
 - 猎物在将来的位置可通过把它的当前速度乘以 T ，并把该偏移量与当前位置相加来得到。
- <http://www.red3d.com/cwr/steer/PursueEvade.html>

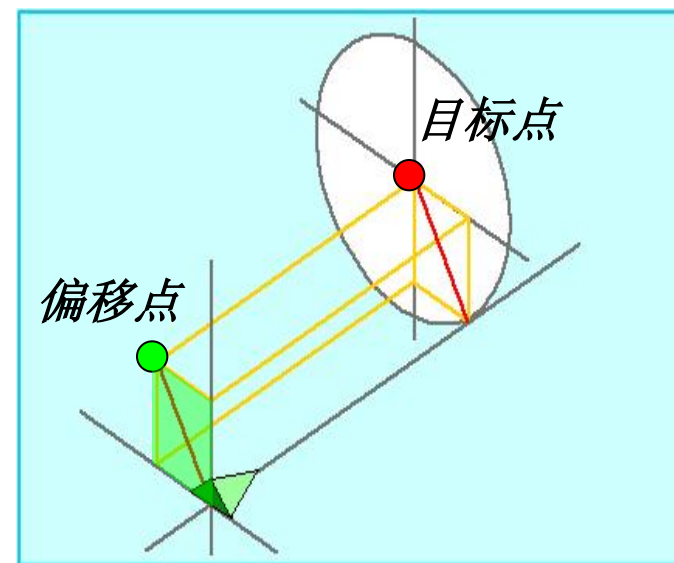
追逐和躲避(Pursuit and Evasion)



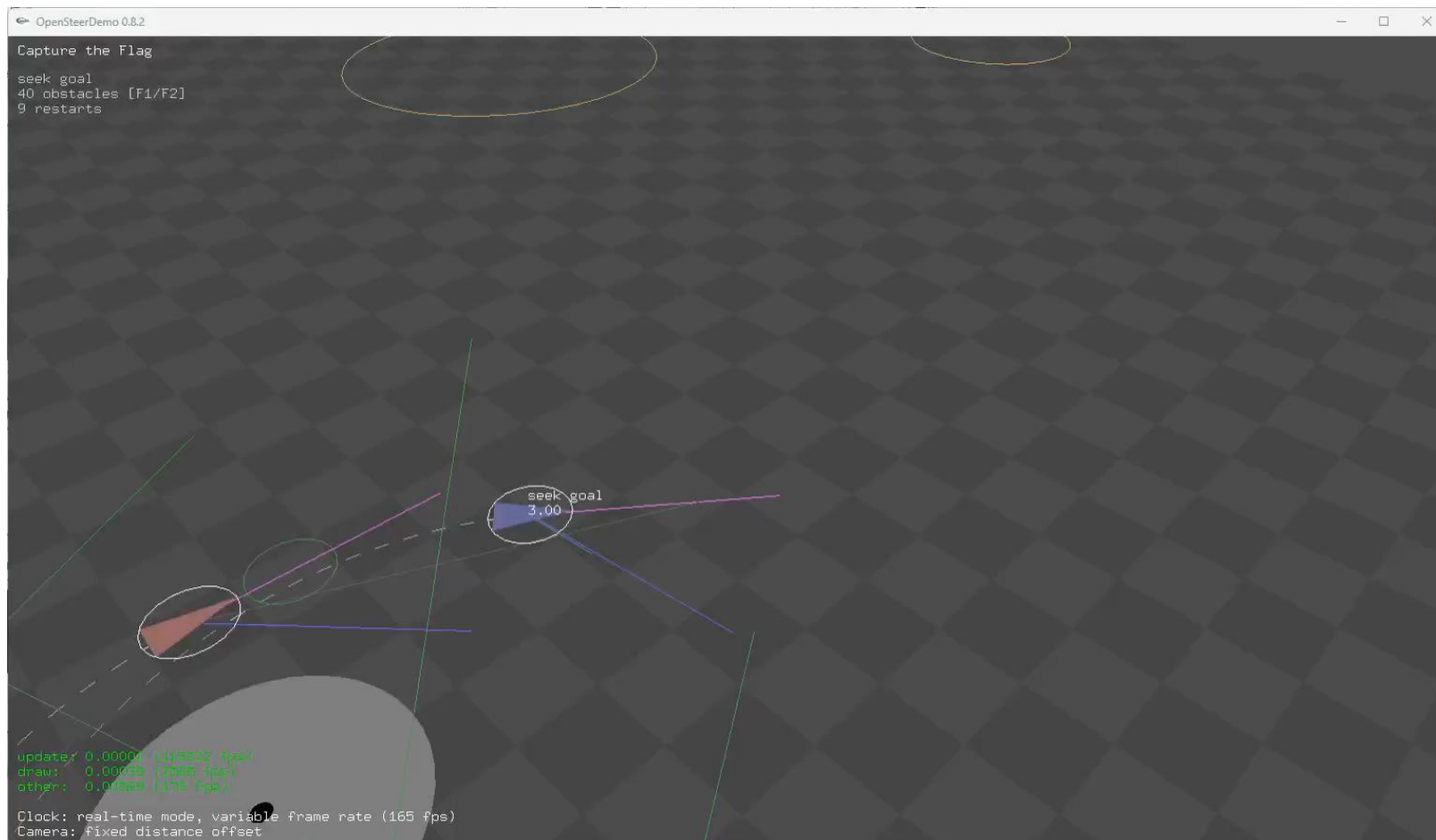
注: 也有把Pursuit and Evasion说成Seek and Flee的

偏移的追逐(Offset Pursuit)

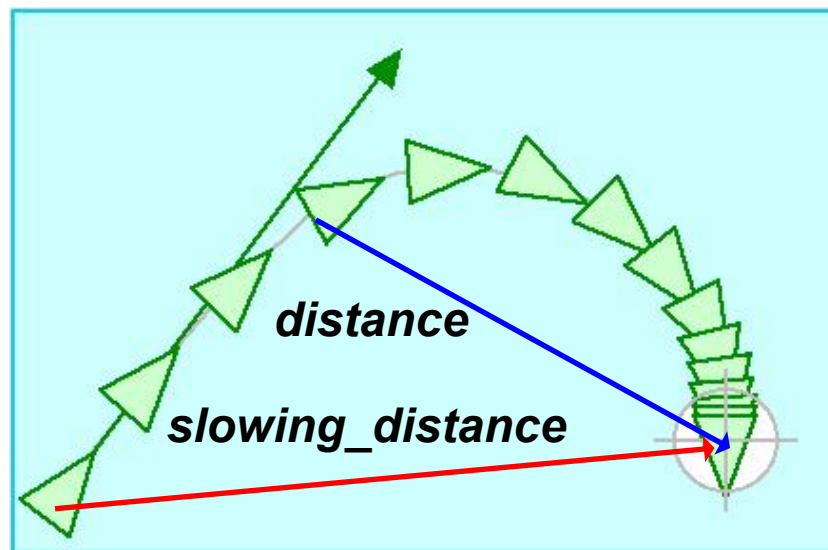
- 沿着一条接近目标的路径追逐，但是不直接进入移动的目标（例子，战斗机进行空中扫射）；
- 对预测的**目标点**进行局部化(在目标角色的局部坐标系)，把局部目标投影到角色的侧向(Side-Up)平面，对偏移向量进行归一化，乘上比例因子R(偏移值)，并把它加到局部目标点, 然后把该值进行全局化(变换回世界坐标系)。
- 采用**寻找**(Seek)行为来接近**偏移点**。



偏移的追逐(Offset Pursuit)



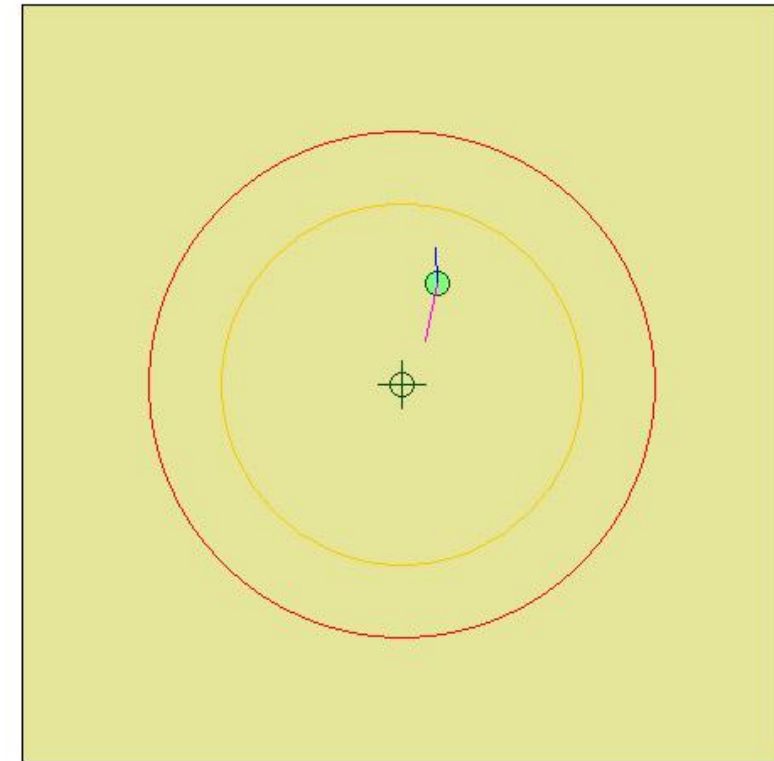
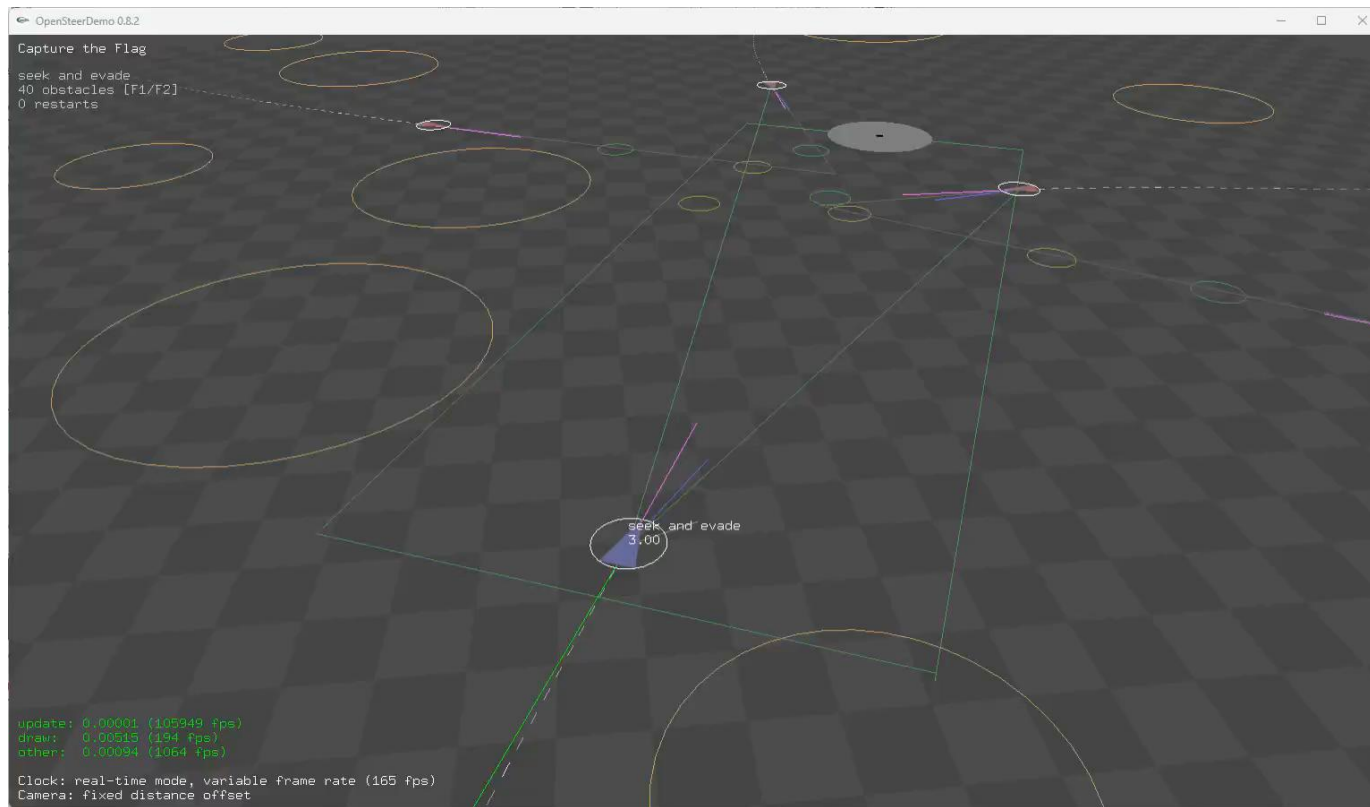
到达(Arrival)



- 当角色远离目标时，到达行为与寻找行为几乎是一样的
- 但是，寻找行为是以全速穿过目标。而对于到达行为，角色将放慢速度，直到停止，并与目标位置重合
- 开始减速的距离为到达行为的一个参数

到达(Arrival)

Arrival steering behavior



到达(Arrival)

- <http://www.red3d.com/cwr/steer/Arrival.html>
- $\text{target_offset} = \text{target} - \text{position}$ (这是一个矢量)

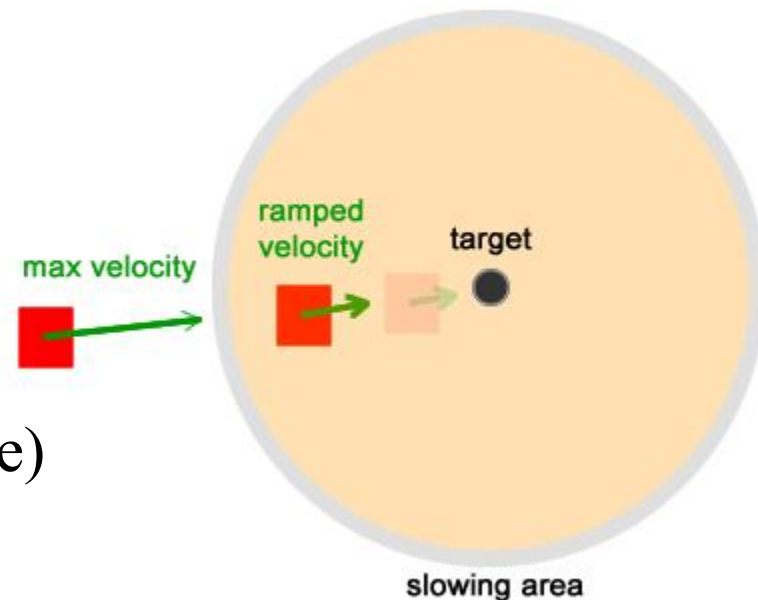
$\text{distance} = \text{length}(\text{target_offset})$ (这是一个标量)

$\text{ramped_speed} = \text{max_speed} * (\text{distance} / \text{slowing_distance})$

$\text{clipped_speed} = \text{minimum}(\text{ramped_speed}, \text{max_speed})$

$\text{desired_velocity} = \text{clipped_speed} * \text{target_offset} / \text{distance}$

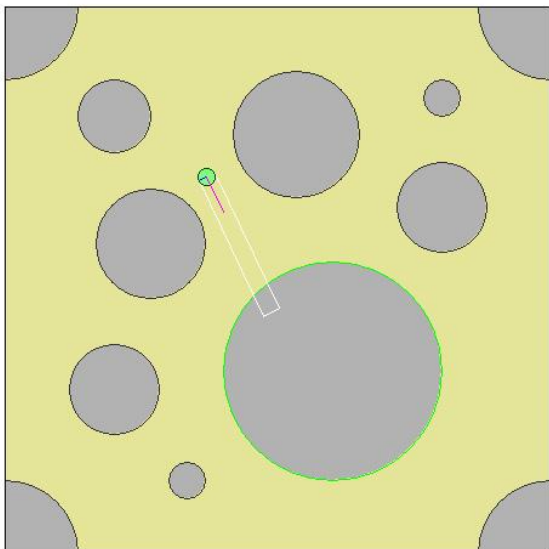
$\text{steering} = \text{desired_velocity} - \text{velocity}$



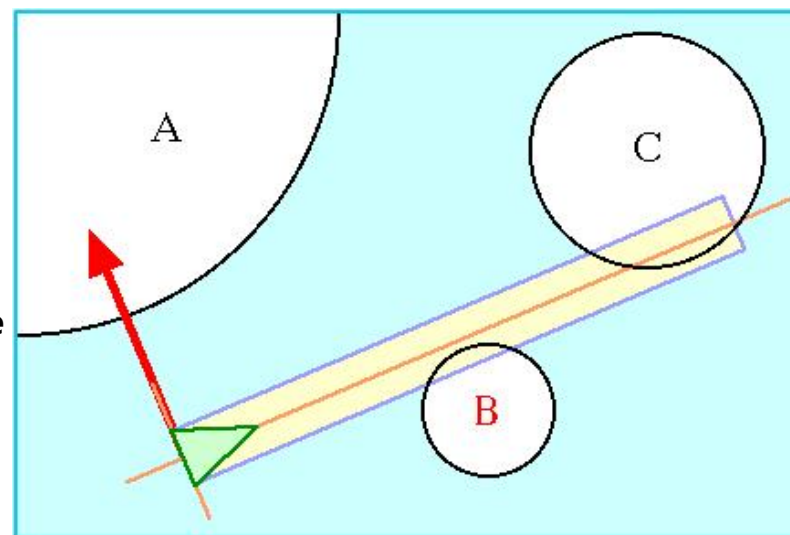
slowing_distance: 表示开始减速的距离

障碍避免(Obstacle Avoidance)

Obstacle Avoidance steering behavior



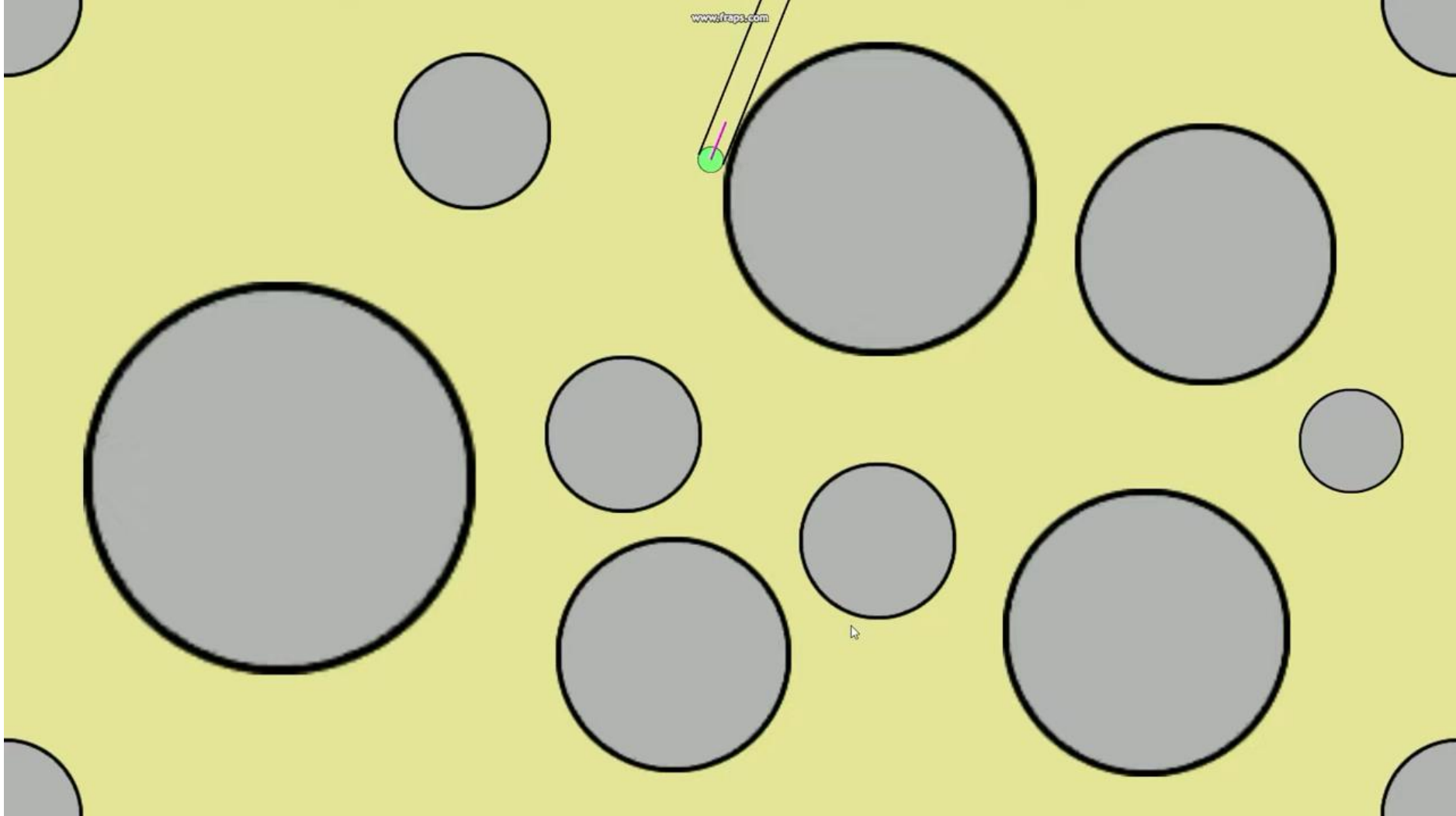
anticipatory collision avoidance



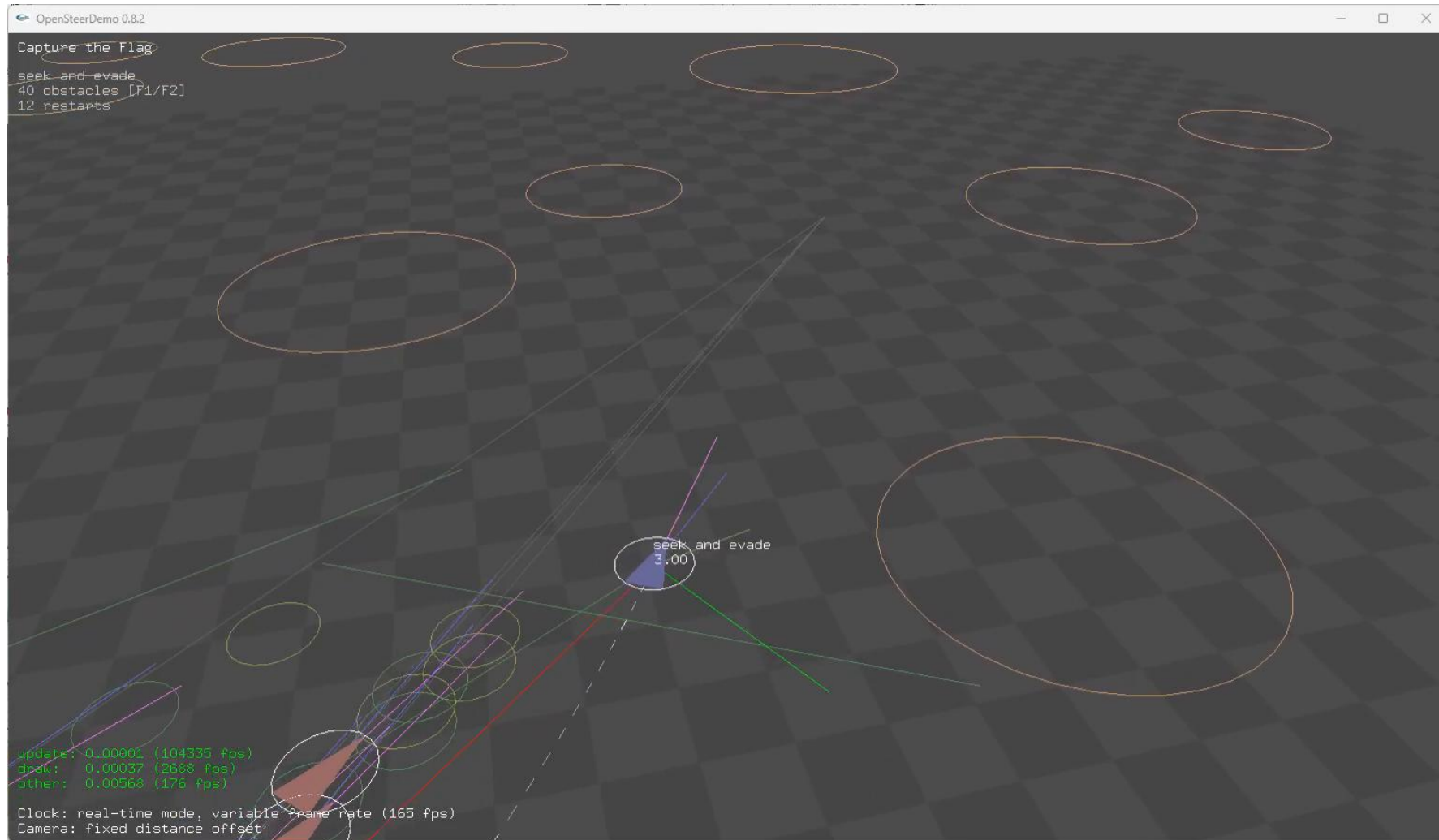
- 该行为的目标是：在该角色前面，保证有一个假想圆柱的自由空间(实际上是预期性碰撞避免)。
- <http://www.red3d.com/cwr/steer/Obstacle.html>

- 障碍避免返回的值：
 - 返回避免最有威胁障碍物的导航值；
 - 如果没有碰撞是迫在眉睫的，返回一个特殊值（空值，或零向量），表示在这一时刻不需要纠正量。

障碍避免(Obstacle Avoidance)



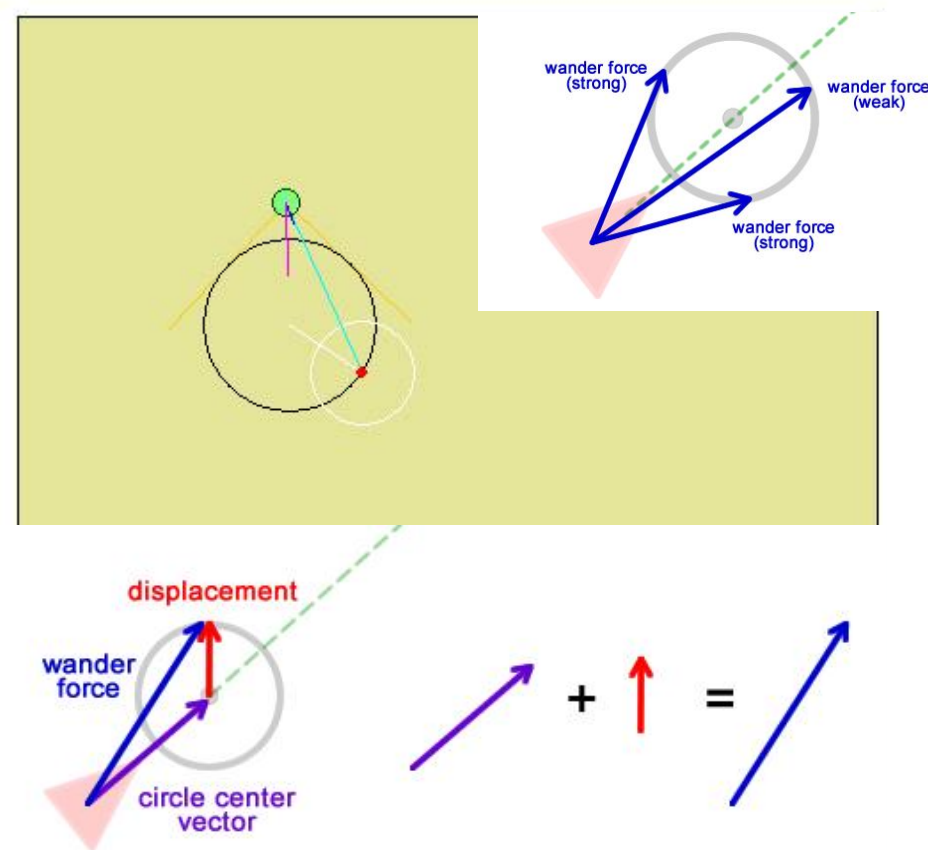
障碍避免(Obstacle Avoidance)



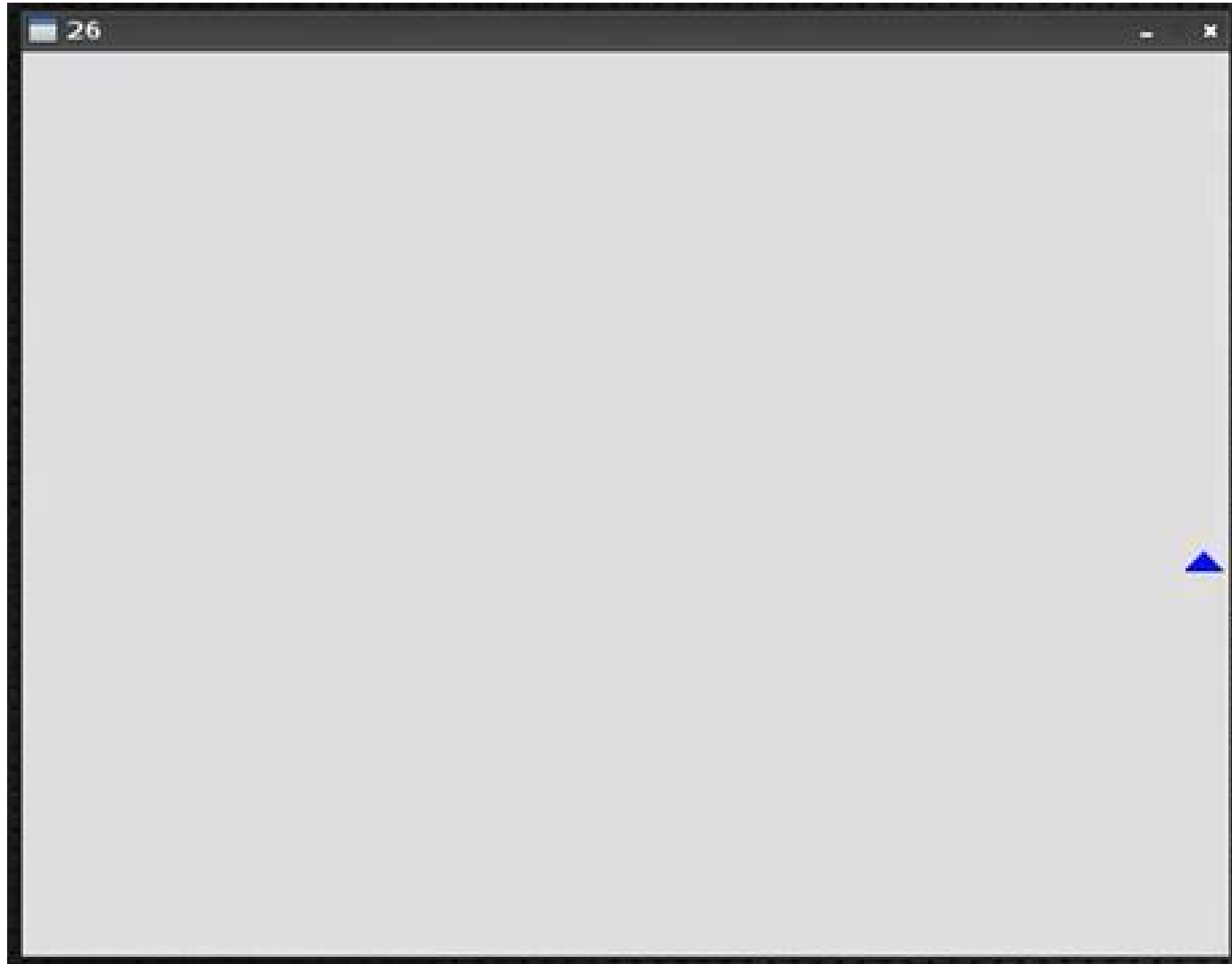
徘徊(Wander)

- 一种随机导航，但又有一定的秩序：某一帧的导航方向与下一帧的导航方向有关。这将产生比生成一随机导航方向更有趣的运动(旨在产生逼真的“随意”动作，让玩家认为角色真的还活着并且四处走动)。
- 徘徊行为将保持一个“徘徊方向”这一状态，这里表示为一个红点。调整的徘徊方向约束到一个黑圆上。黑圆的半径为1，其中心处于沿角色向前轴(*forward axis*) $\sqrt{2}$ 单位长度的位置。
- 徘徊模型的参数:
 - **Strength**: 从0(没有转向) 到 1.
 - **Rate**: 控制 “徘徊方向”变化的程度，从0 (没变化) 到1 (快速的飘忽不定的变化)，这里表示为白圆的半径 (上图显示的为 0.6，表示最大的随机偏移量)
- **导航方向**表示为青色的矢量。车辆的速度用品红矢量来表示，导航力用蓝色矢量来表示。

Wander steering behavior



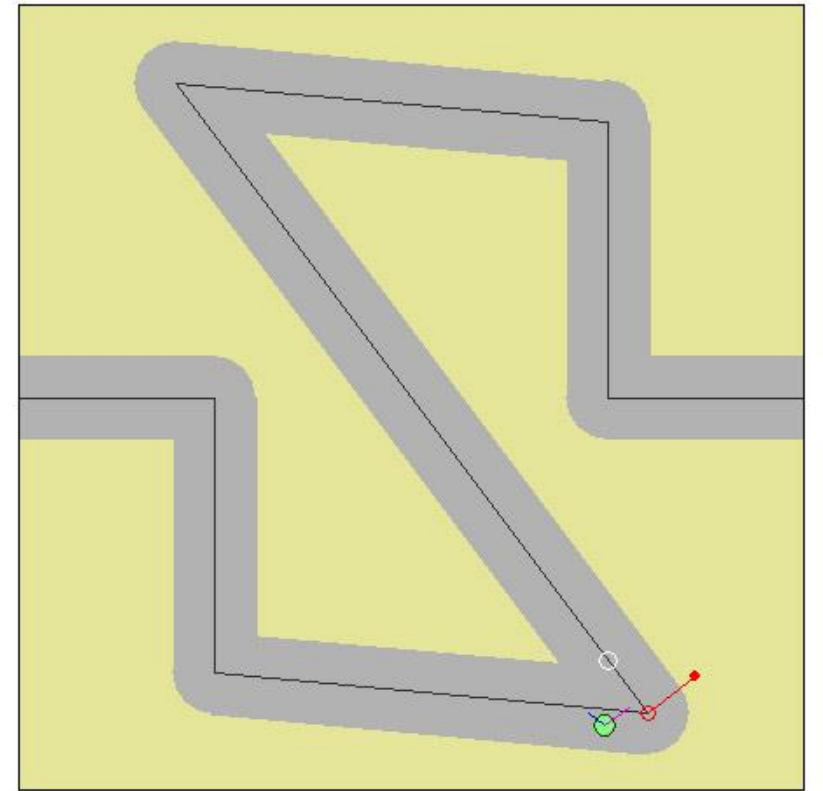
徘徊(Wander)



路径跟随(Path following)

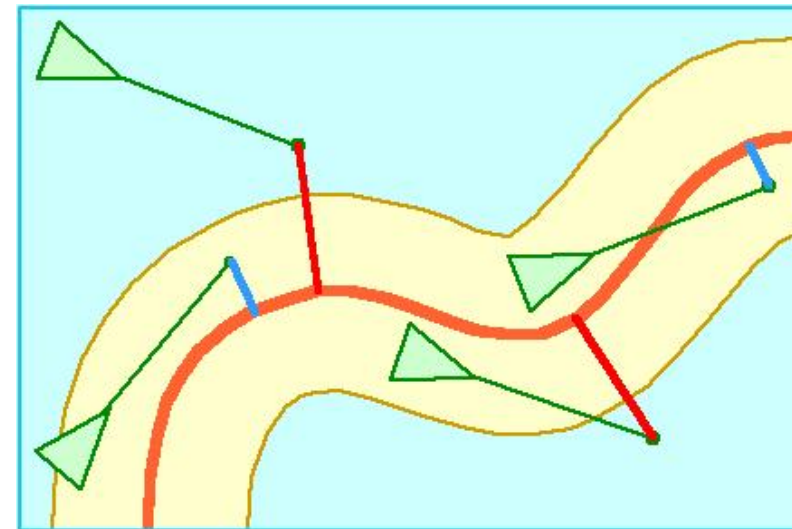
- 绿圆显示了路径跟随行为。其任务是在一个给定的方向进行路径遍历（左边进入，右边退出），同时保持其中心位于灰色区域。
- 在该例子中，路径定义为一条折线和一个半径。该行为与抑制(containment)和沿墙运动(wall following) 相关，其区别在于，在路径跟随中，路径意味着行驶方向；
- 在路径跟随行为中，只有当车辆将离开灰色区域时，才执行校正导航。

Path Following steering behavior



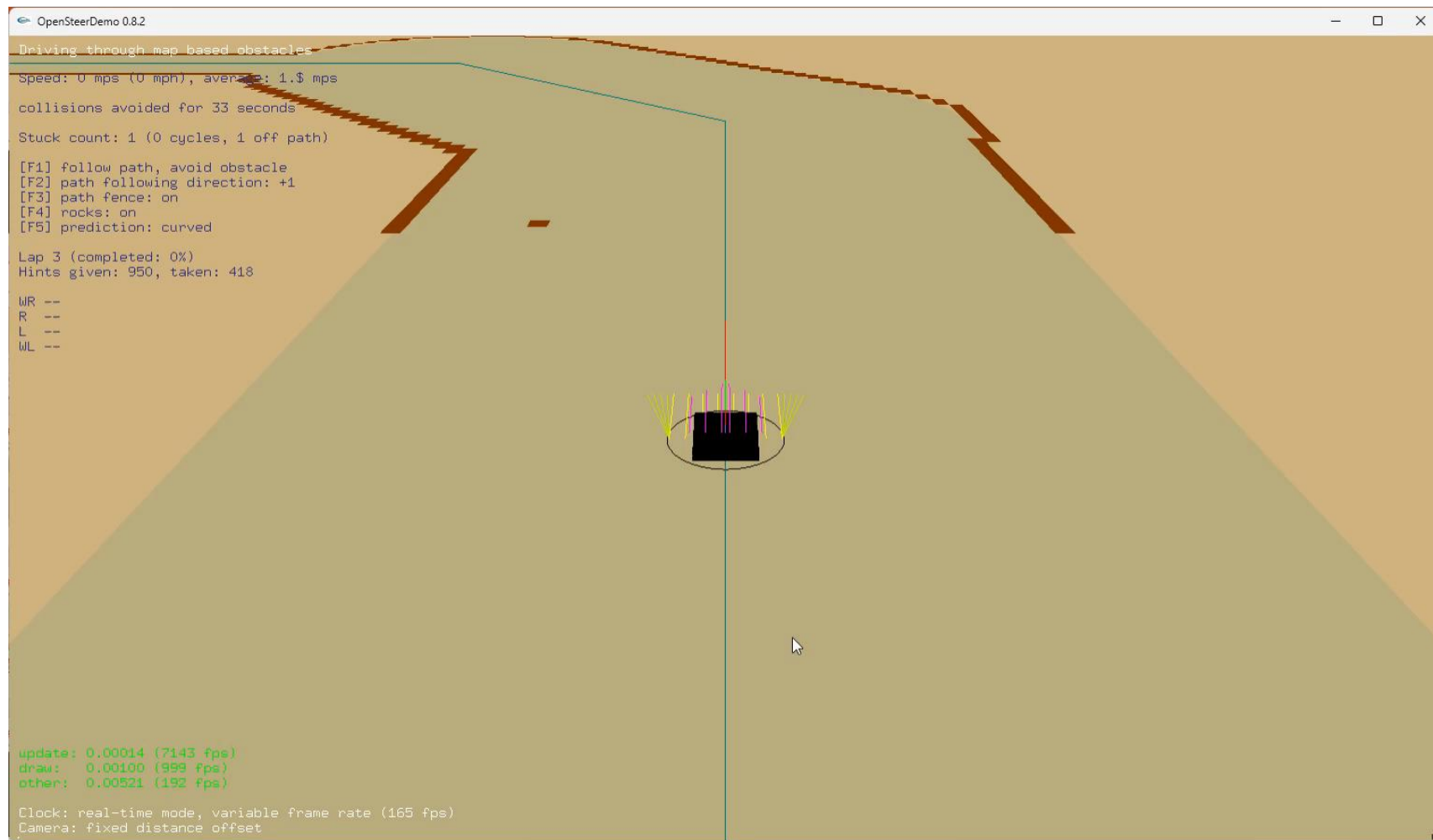
路径跟随(Path following)

- <http://www.red3d.com/cwr/steer/PathFollow.html>
- 沿着一条路径移动角色，并同时保持在脊柱线的指定半径内。
- **投影距离**：小于路径半径时，不需要进行导航校正。
- 否则，把预测的位置投影到路径上，把该点作为目标点，并进行寻找(Seeking)行为。



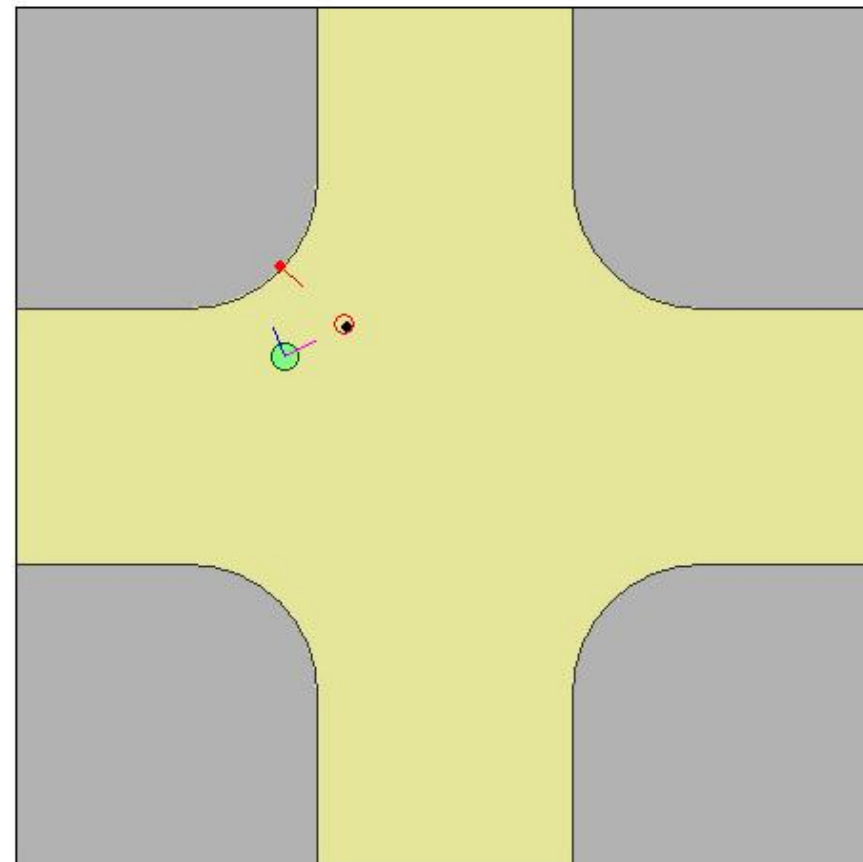
投影距离(Projection distance) :
从预测位置到最近路径点的距离

路径跟随(Path following)



沿墙运动(Wall Following)

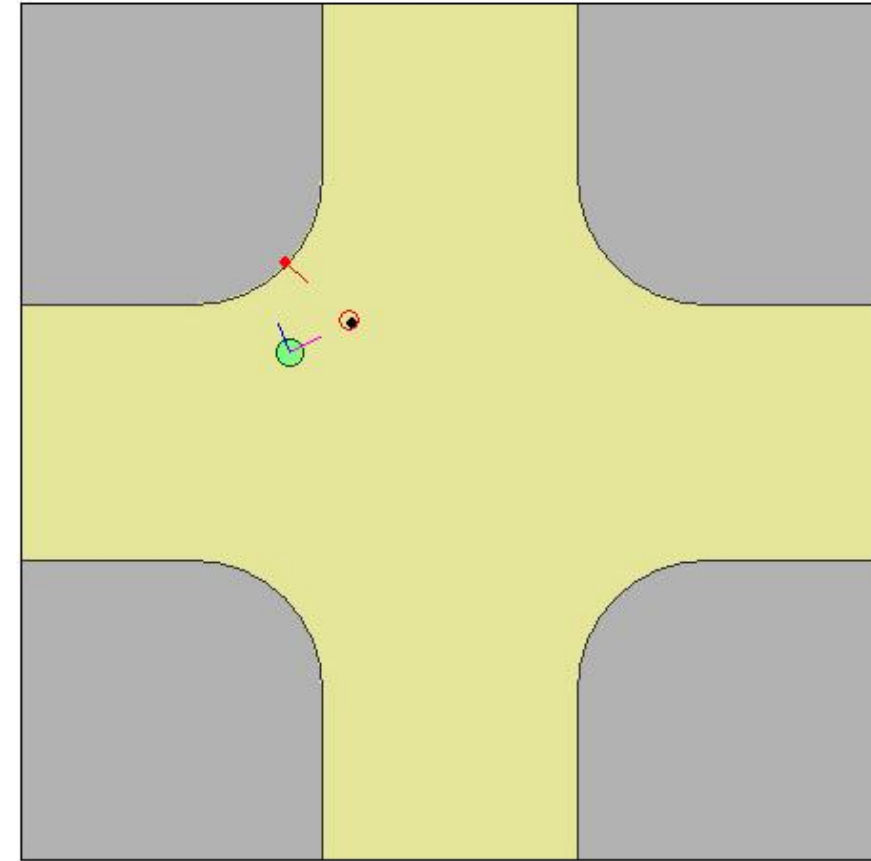
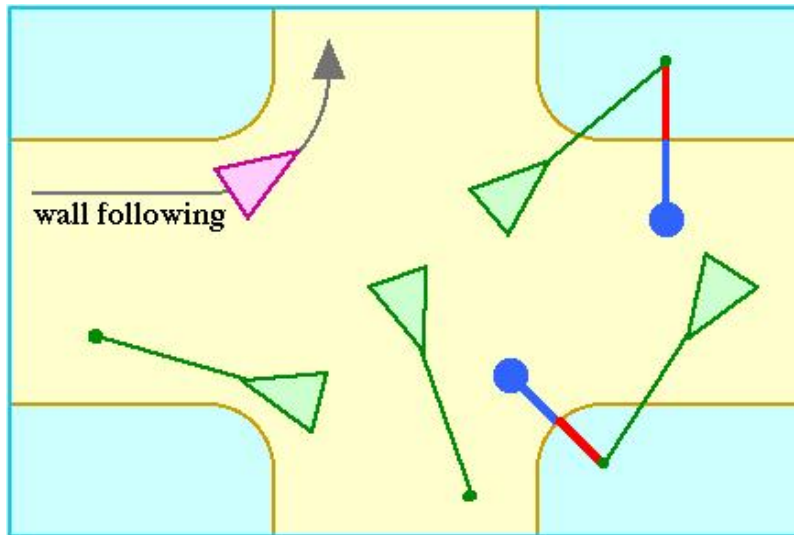
- 绿色车辆使用沿墙运动行为以使得它沿标记为灰色的区域平行移动并保持一定的偏移量。汽车在移动时，需要保持与墙给定的距离。车辆的速度由品红色矢量表示，导航力由一蓝色矢量表示。
- 沿墙运动行为的实现：首先根据当前的速度预测一定时间后车辆的位置，把该位置投影到墙上离它最近的点(红点)，把该点沿墙的法向(红线)移动给定的偏移量，从而生成一个目标点(红色的圆)。最后采用寻找(Seek)行为来接近目标点。



<http://www.red3d.com/cwr/steer/Wall.html>

沿墙运动(Wall Following)

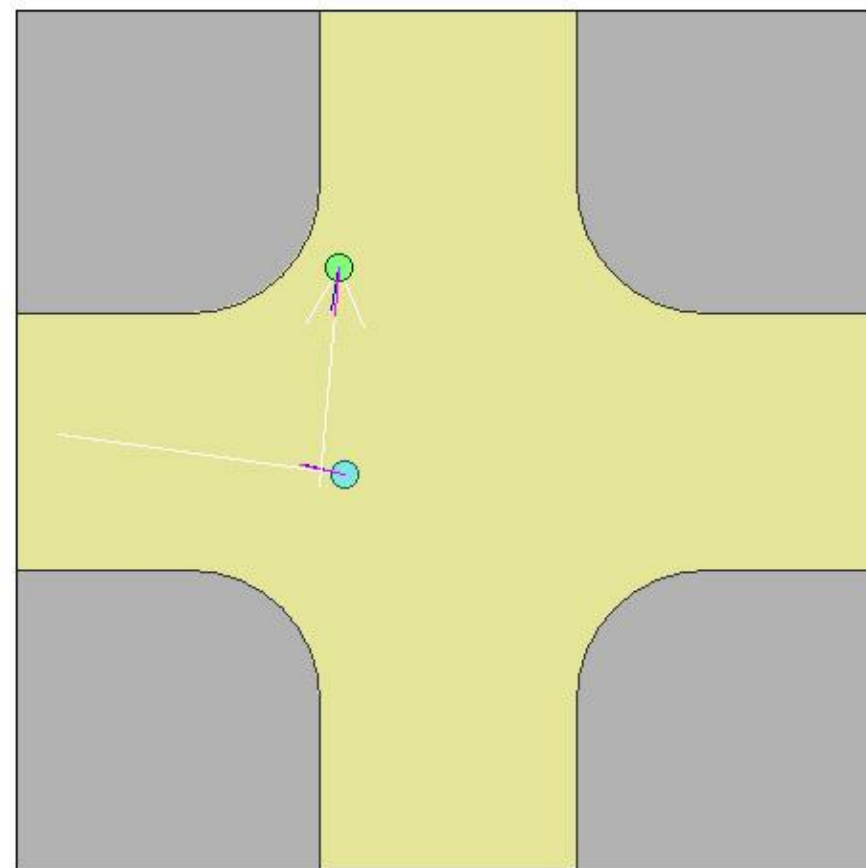
- 车辆与墙之间的通信通过一个通用的表面协议 (generic surface protocol)来实现：车辆经过墙表面时，经过询问可得到墙上的最近点和该点的法向。因此，导航行为并不需要知道墙壁表面的形状信息。



抑制 (Containment)

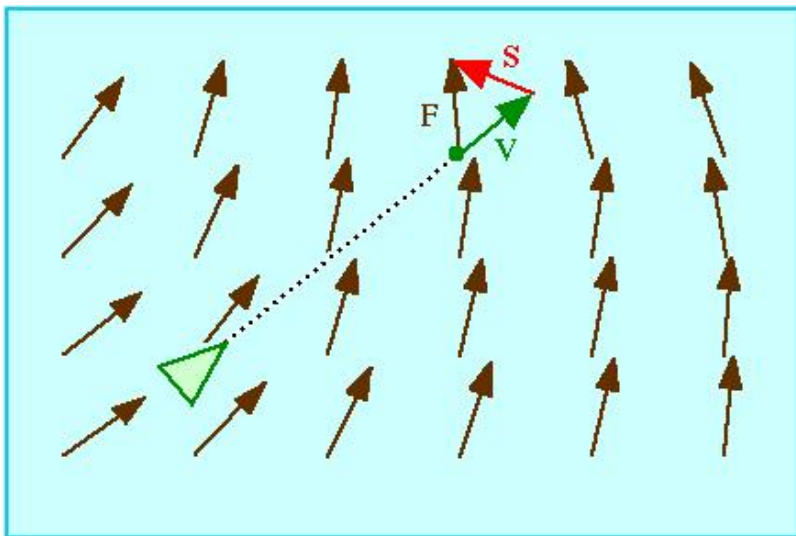
- <http://www.red3d.com/cwr/steer/Containment.html>
- 车辆使用抑制(或广义的障碍避免)行为使得它远离标记为灰色的区域。这种行为与前述的障碍避免密切相关（障碍物通过包围球或圆柱表示）。然而，这种行为可以处理任意形状，并允许车辆导航接近障碍物的表面。
- 车辆通过探测点测试前面的空间(用白线表示)，当没有接触障碍物时，车辆将采用徘徊行为，以避免使得它的运动与“通道”对齐。当探测点接触到障碍物时，把该点投影到障碍物的表面，并计算这点的法向，沿法向偏移一定量的点作为Seek目标进行导航。
- 车辆与障碍物之间的通信通过一个通用的表面协议(generic surface protocol)来实现：车辆询问探测点是否位于障碍物内部，如果是的话，得到障碍物上的最近点和该点的法向。因此，导航行为并不需要知道墙壁表面的形状信息。

Containment steering behavior



流场跟随行为(Flow Field Following)

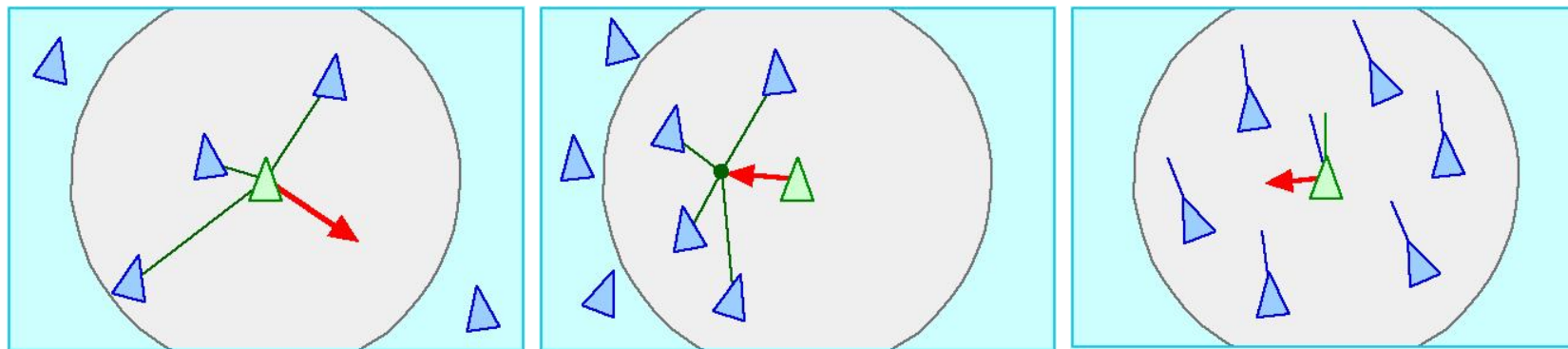
- <http://www.red3d.com/cwr/steer/FlowFollow.html>
- 假设运动区域已经有一个速度流场
- 估算角色将来的位置并计算在这点的流场
- 得到的速度(F)即为我们期望的速度，导航方向(S)为期望速度和当前速度的差



Flow Field Pathfinding

群组的行为(Group Behavior)

- 分离(Separation)、聚集(cohesion)和对齐(alignment)与群体的行为有关;
- 导航行为决定一个角色如何对周围的角色做出反应;

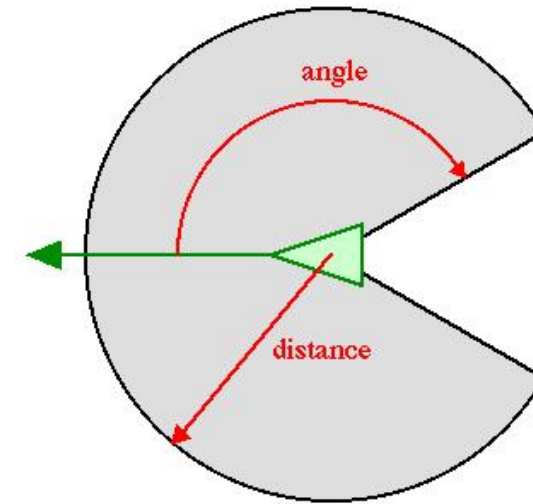


Repulsive force is computed by subtracting the positions of our character and the nearby character

平均速度为我们期望的速度

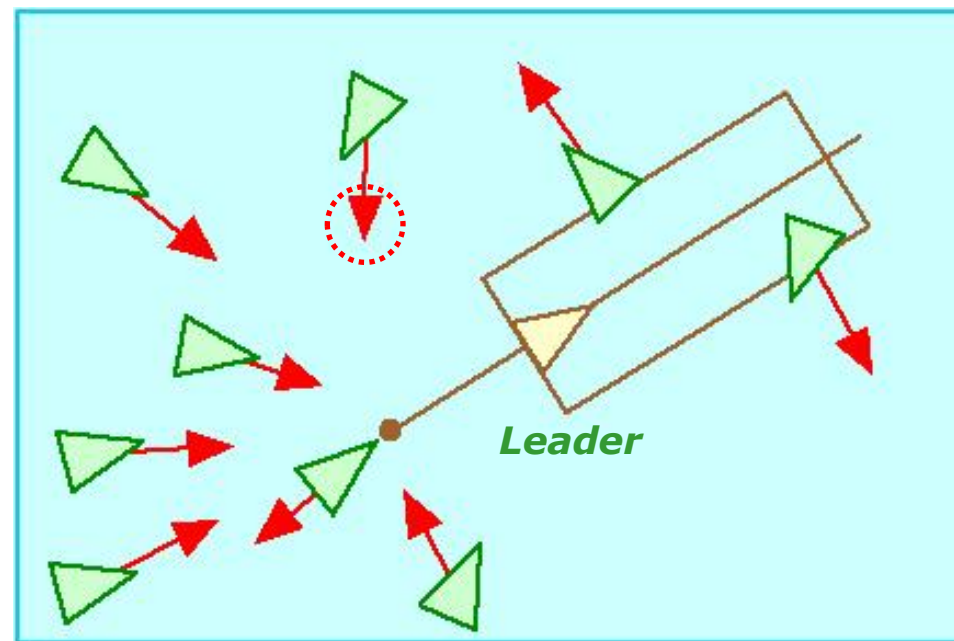
邻居(Neighborhood)

- 角色的局部邻居
 - 邻居群组成员由一个距离 (从角色的中心开始度量)和一角度(从角色的运动方向开始度量)来刻画, 该角度定义了角色的感知视野;



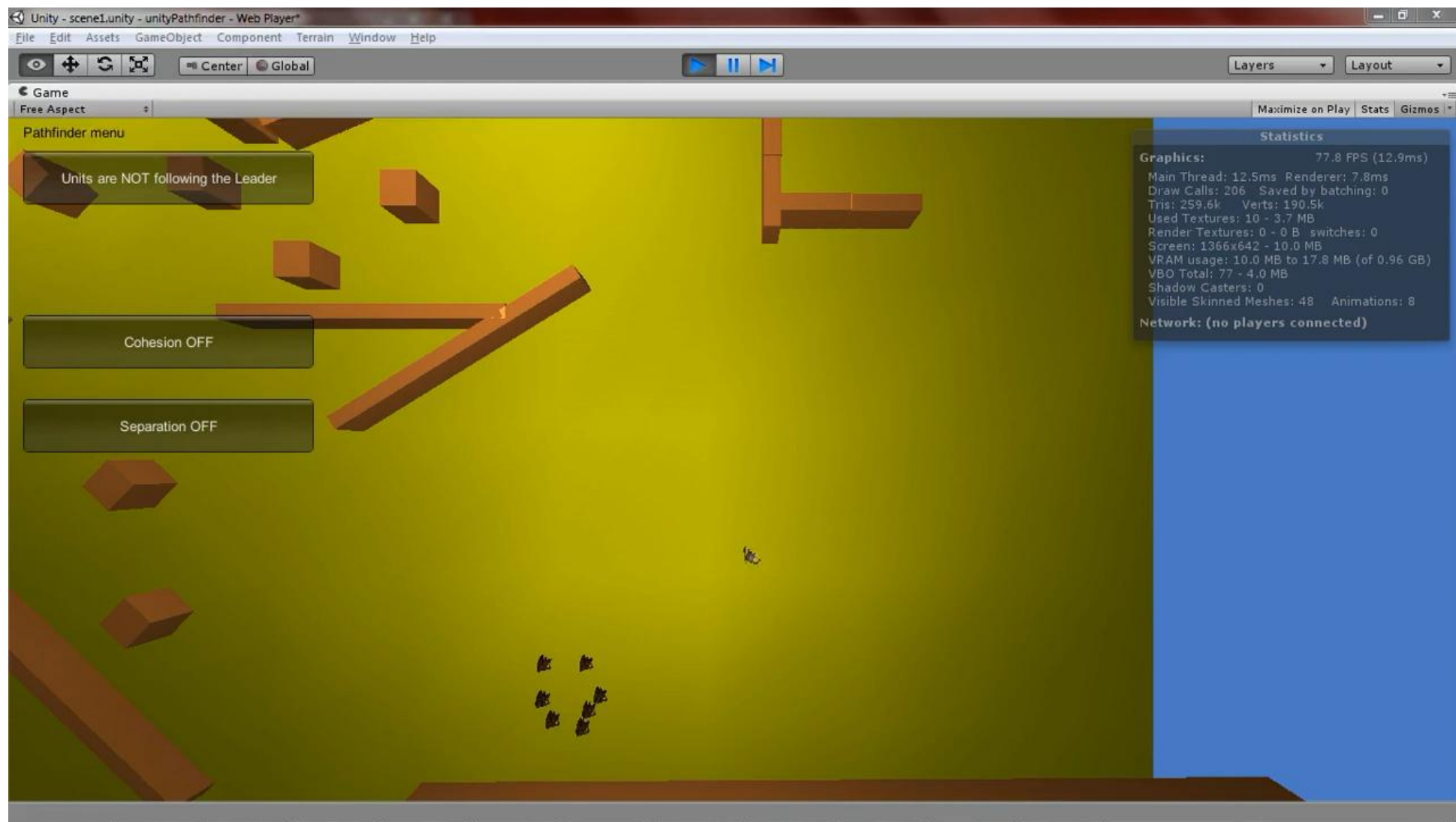
跟随领导(Leader Following)

- 如果一个跟随成员发现自己处于领导前面的一个矩形区域，它会横向远离领导者的路径；
- 否则，到达(arrival)目标为领导后面的一个偏移点；
- 跟随成员采用分离行为来避免相互拥挤；



<http://www.red3d.com/cwr/steer/LeaderFollow.html>

跟随领导(Leader Following)



行为的组合(Combining Behaviors)

- 组合行为的两种方式:
 - 切换(Switch)
 - 混合(Blending)
- 最直接的方法可简单地计算每个组的导航行为并把他们加权求和

TECNICAS PROCEDURALES
EN ANIMACION
ENTREGA 1

总结

- Boids模型：类似鸟集群运动模型
(空中)
- Steering Behaviors for Autonomous Characters：地面自主角色模型的导航

- 方法缺陷：

- 运动可能会不太自然
 - 运动像一个完美的机器人
- 碰撞避免后
 - 一遍又一遍沿着障碍物转
- 碰撞反应
 - 碰撞避免有可能失败



关于Craig W. Reynolds

(born March 15, 1953)

- <http://www.red3d.com/cwr/>
- Winner of a Scientific And Engineering Award presented by The Academy of Motion Picture Arts and Sciences for *pioneering contributions to the development of three dimensional computer animation for motion picture production* at the Scientific and Technical Awards of the 70th Academy Awards® held in 1998.
- 1998 to present
 - Sony Computer Entertainment America
- 1997 to 1998
 - DreamWorks Animation



OpenSteer

Steering Behaviors for Autonomous Characters

- OpenSteer is a C++ library to help construct steering behaviors for autonomous characters in games and animation. In addition to the library, OpenSteer provides an OpenGL-based application called OpenSteerDemo which displays predefined demonstrations of steering behaviors. The user can quickly prototype, visualize, annotate and debug new steering behaviors by writing a plug-in for OpenSteerDemo.
- OpenSteer provides a toolkit of steering behaviors, defined in terms of an abstract mobile agent called a "vehicle." Sample code is provided, including a simple vehicle implementation and examples of combining simple steering behaviors to produce more complex behavior. OpenSteer's classes have been designed to flexibly integrate with existing game engines by either layering or inheritance.

基于社会力模型的群体行为模拟



- 参考文献

D. Helbing, I. Farkas, T. Vicsek: Simulating dynamical features of escape panic, *Nature* 407: 487 (2000) (Google citations: 6888次, 2026.4)

Stampedes (踩踏事件)

- Videos and descriptions of disasters

This is a partial list of human stampedes. The following are some well-known disasters:

24.07.2010, Love Parade, Duisburg (Germany), [Wikipedia](#), [YouTube](#)

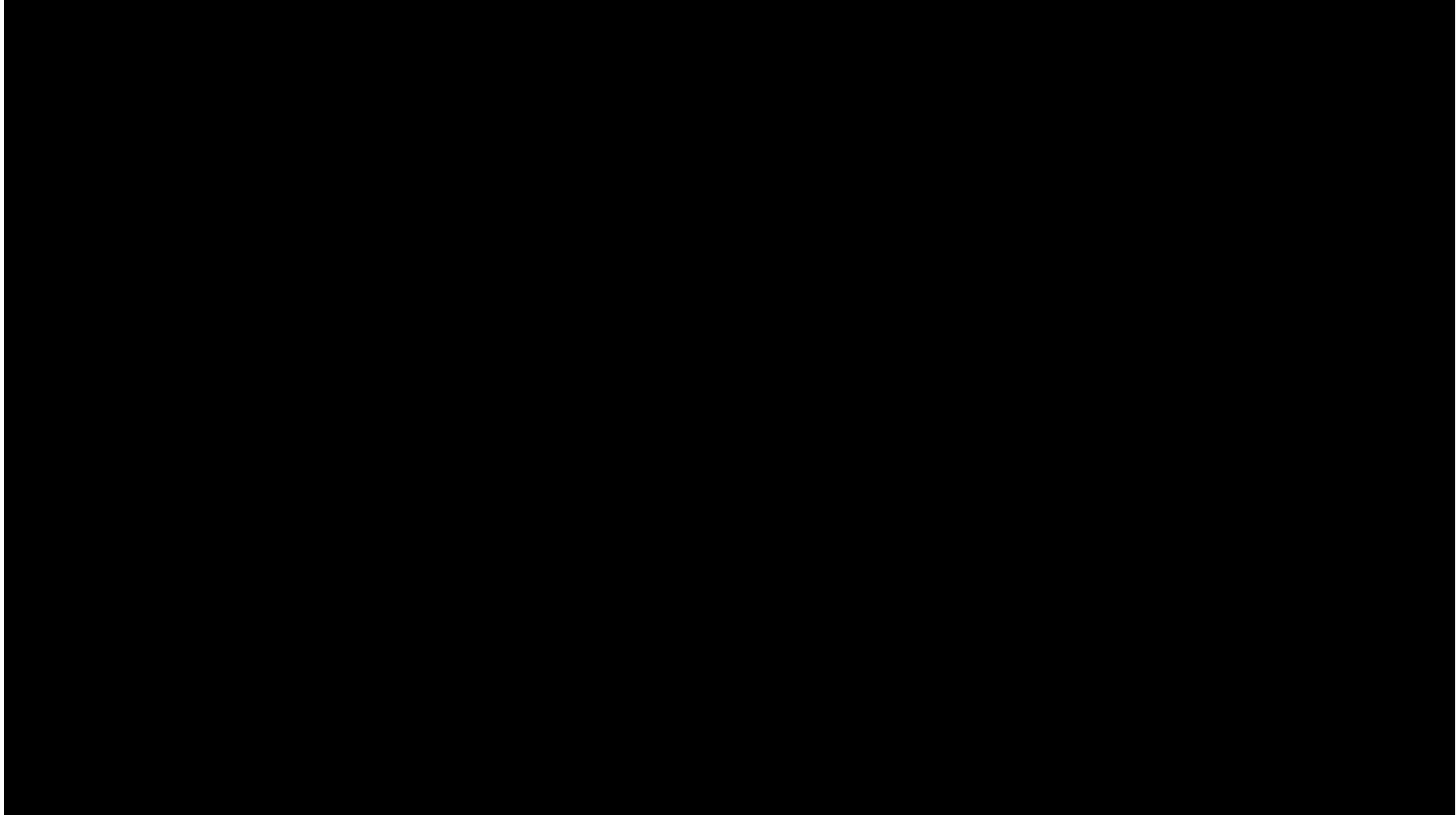
15.04.1989, Hillsborough, Sheffield (UK), [Wikipedia](#), [YouTube](#)

29.05.1985, Heysel stadium, Brussels (Belgium), [Wikipedia](#), [YouTube](#)

02.01.1971, Ibrox (UK), [Wikipedia](#), [YouTube](#)

- Website: <http://angel.elte.hu/panic>

Duisburg (Germany) 2010



问题提出

- 人群的**羊群效应**可以是致命的;
- 当处于恐慌状态时, 人的行为动作会变得笨拙和危险;
- 公共场合中的人群聚集通常是事故发生的隐患, 人群拥挤度越高, 事故风险也就越大, 而且造成的人员伤亡和损失也更为严重;
- 获取真实的人群恐慌行为数据是相当困难的;
- 建立一个有效的行人群体行为仿真模型对公共安全, 降低事故风险有着重要的意义。

解决方法

- Helbing等提出了**社会力模型**的概念，并对人群运动进行了仿真，展现了人群疏散过程中的堵塞现象。
- 把人抽象为自我驱动的质点；
- 把人群运动中的物理和社会心理学影响用社会力来进行建模；
- 模拟人群的恐慌现象并看看会发生什么。

基本原理

- 社会力模型以牛顿动力学为基础，由各个力的表达式来体现行人不同的动机和影响。在**社会力模型**中，由于对影响个体的因素考虑得比较全面，对个体行为的建模比较合理，该模型可以逼真地模拟人群的疏散过程。
- 个体的实际行为受**主观意识**、**其他个体**及**障碍物**三方面因素的影响，均可等效为**力**在个体上的作用。

仿真人群的牛顿动力学模型

$$m_i \frac{d\mathbf{v}_i}{dt} = m_i \underbrace{\frac{v_i^0(t)\mathbf{e}_i^0(t) - \mathbf{v}_i(t)}{\tau_i}}_{\text{目标驱动力}} + \sum_{j(\neq i)} \mathbf{f}_{ij} + \sum_w \mathbf{f}_{iw}$$

其他个体对*i* 边界力
产生的力

$v_i(t)$ = 行人*i*的实际速度

1. 个体*i*的自身驱动力

- 主观意识对个体行为的影响可等效为个体受自身施加的“**社会力**”，体现了行人以期望的速度移动到目的地的动机：

$$\mathbf{f}_i^0(t) = m_i \frac{v_i^0(t) \mathbf{e}_i^0(t) - \mathbf{v}_i(t)}{\tau_i}$$

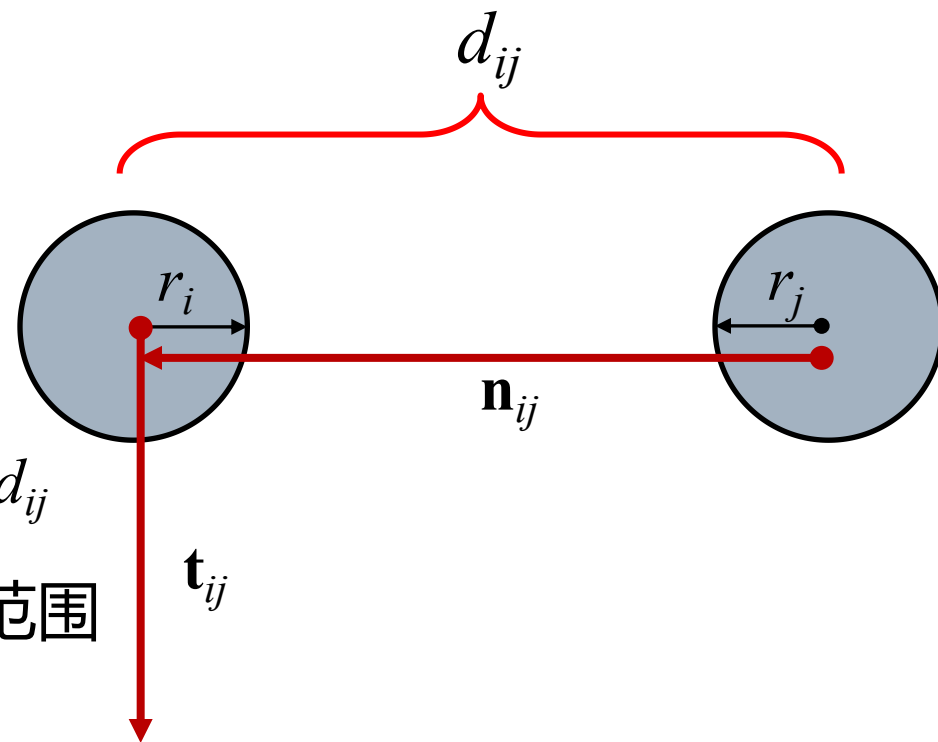
m_i 为行人质量， $v_i^0(t)$ 为行人的期望速度， $\mathbf{e}_i^0(t)$ 为行人的期望运动方向， τ_i 为调整时间常数。

- 期望的运动方向可由行人此刻的位置 $\mathbf{r}_i$ 和目标位置 $\mathbf{p}$ 算出， 即 $\mathbf{e}_i^0(t) = \frac{\mathbf{p} - \mathbf{r}_i}{\|\mathbf{p} - \mathbf{r}_i\|}$

2. 个体*j*对个体*i*产生的力

$$\mathbf{f}_{ij} = \left\{ A_i \exp \left[\left(r_{ij} - d_{ij} \right) / B_i \right] + kg \left(r_{ij} - d_{ij} \right) \right\} \mathbf{n}_{ij} + \kappa g \left(r_{ij} - d_{ij} \right) \Delta v_{ji}^t \mathbf{t}_{ij}$$

- $A_i \exp \left[\left(r_{ij} - d_{ij} \right) / B_i \right] \mathbf{n}_{ij}$ 为**心理学排斥力**
- r_{ij} 为个体*i*和个体*j*的半径之和
- d_{ij} 为个体*i*和个体*j*中心之间的距离
- $\mathbf{n}_{ij}$ 为个体*j*到个体*i*的单位矢量 $\mathbf{n}_{ij} = (n_{ij}^1, n_{ij}^2) = (\mathbf{r}_i - \mathbf{r}_j) / d_{ij}$
- A_i 和 B_i 为常数, 表示相互作用的强度和调节作用范围



2. 个体*j*对个体*i*产生的力

- $kg(r_{ij} - d_{ij})\mathbf{n}_{ij}$ 个体*j*对个体*i*施加的**身体力**，是行人为了避免身体接触避让他人所产生的力(*k*为一很大的常数):

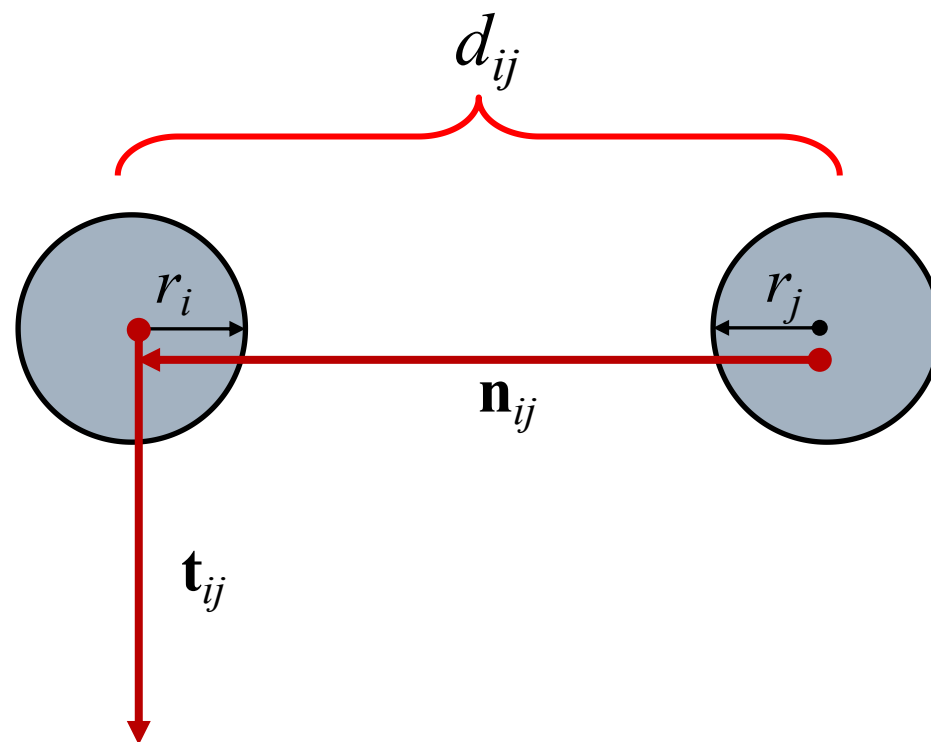
$$g(x) = \begin{cases} x; & x > 0 \\ 0; & x \leq 0 \end{cases}$$

- 当*x*>0时，表示两个行人之间的半径之和大于两者之间的距离，两人之间**有身体接触**;
- 当*x*<=0时，两个行人之间**没有身体接触**，两人之间的身体力为0;

2. 个体*j*对个体*i*产生的力

$\kappa g(r_{ij} - d_{ij}) \Delta v_{ji}^t \mathbf{t}_{ij}$ 为行人之间的**滑动摩擦力**

- $\mathbf{t}_{ij} = (-\mathbf{n}_{ij}^2, \mathbf{n}_{ij}^1)$ 为切向方向
- Δv_{ji}^t 为切向方向的速度差
- κ 为一个很大的常数



3. 环境力

- 边界和障碍物对人的影响类似于人与人之间的作用。边界 w 对个体 i 的影响为：

$$\mathbf{f}_{iw} = \left\{ A_i \exp \left[(r_i - d_{iw}) / B_i \right] + kg(r_i - d_{iw}) \right\} \mathbf{n}_{iw} - \kappa g(r_i - d_{iw}) (\mathbf{v}_i \bullet \mathbf{t}_{iw}) \mathbf{t}_{iw}$$

□ d_{iw} 为个体 i 到环境 w 之间的距离

□ $\mathbf{t}_{iw}$ 切矢量

羊群效应

- 恐慌的人群呈现羊群效应、个体行为或两者的混合；
- 羊群效应用**恐慌参数** p_i 来模拟；

$$\mathbf{e}_i^0(t) = \text{Norm} \left[(1 - p_i) \mathbf{e}_i + p_i \left\langle \mathbf{e}_j^0(t) \right\rangle_i \right]$$



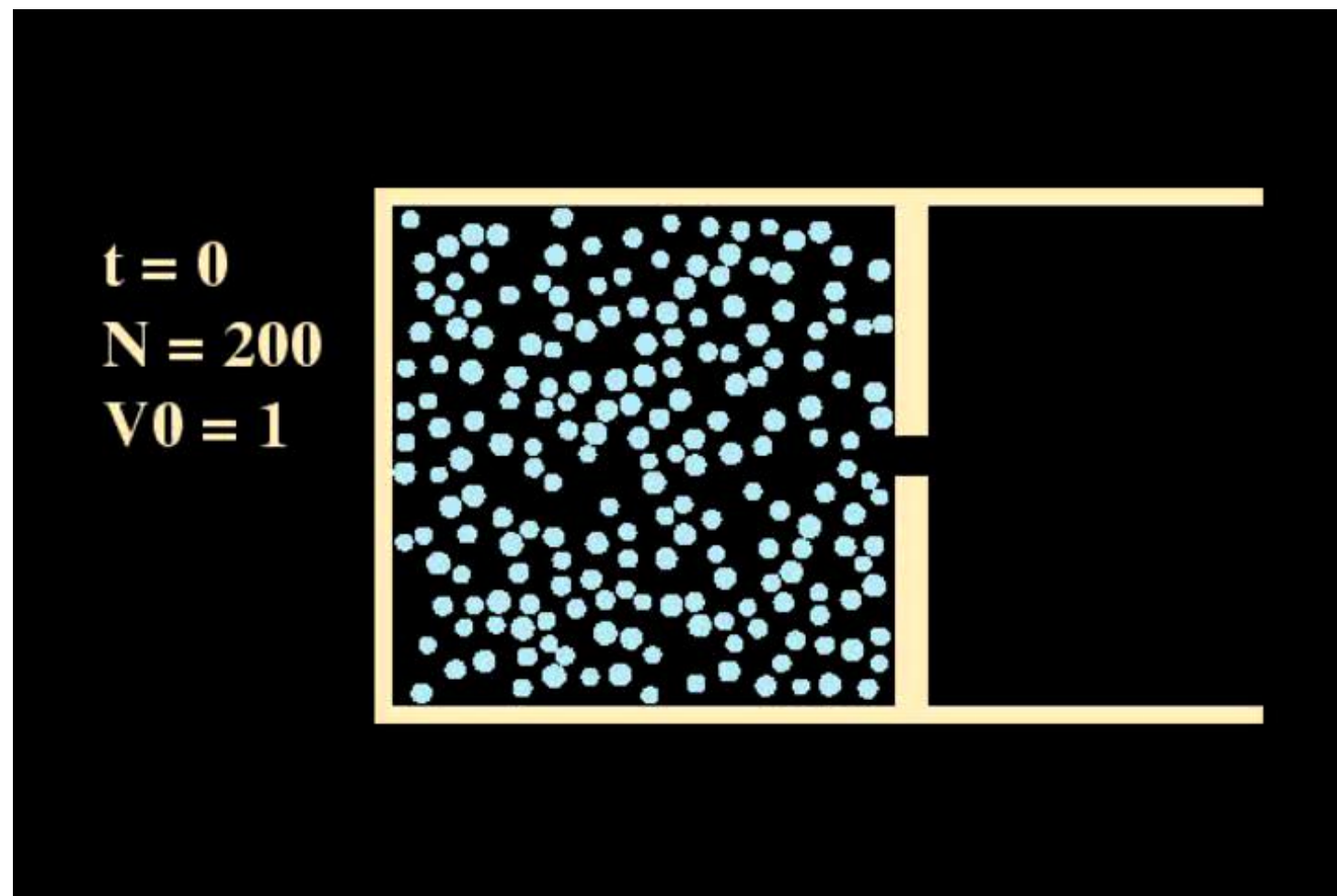
个体行为的方向



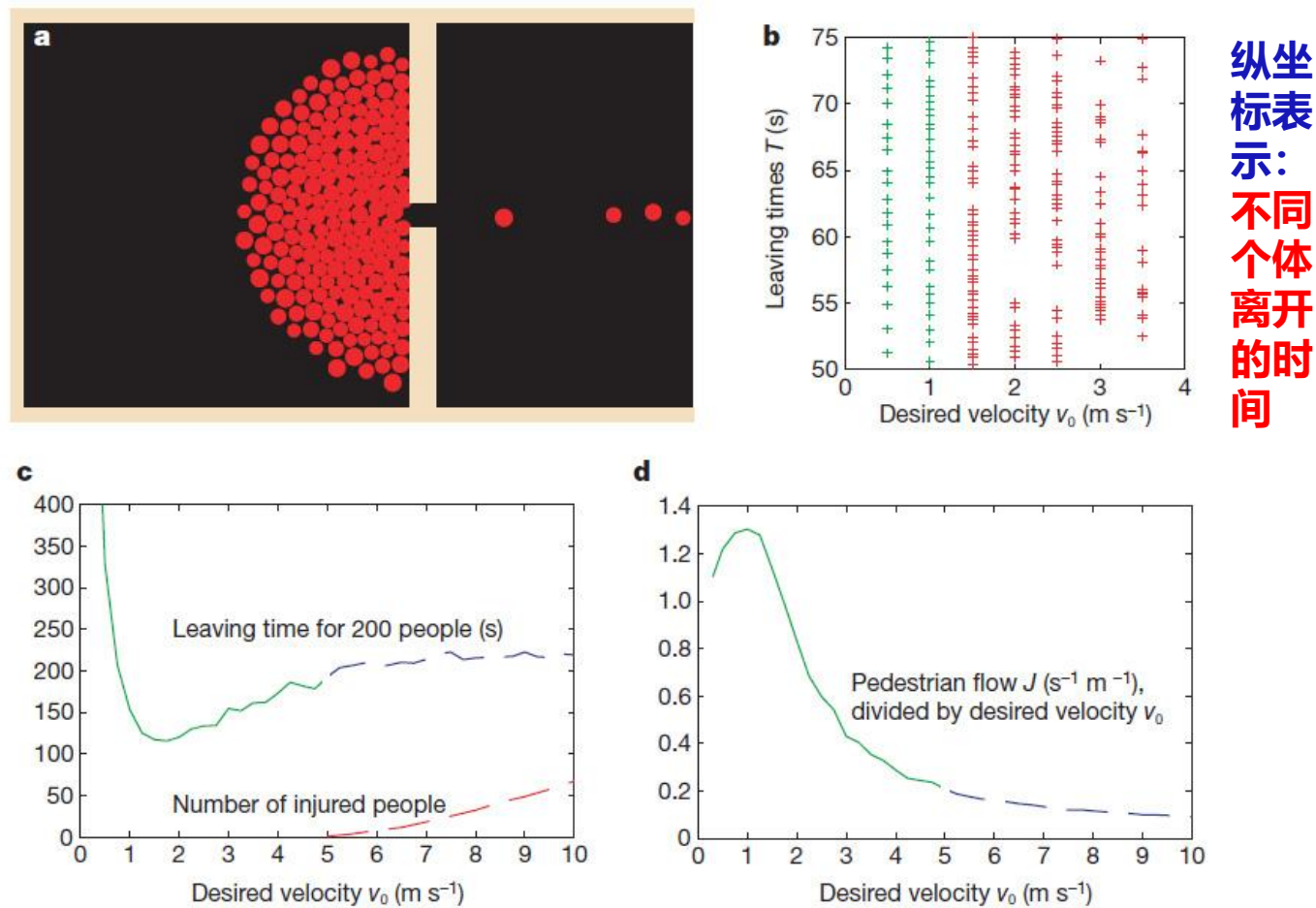
个体邻居行为的平均

常量和参数的选择

- 根据非恐慌状态下人流的状况来选择参数
- 行人用半径不同的圆来描述

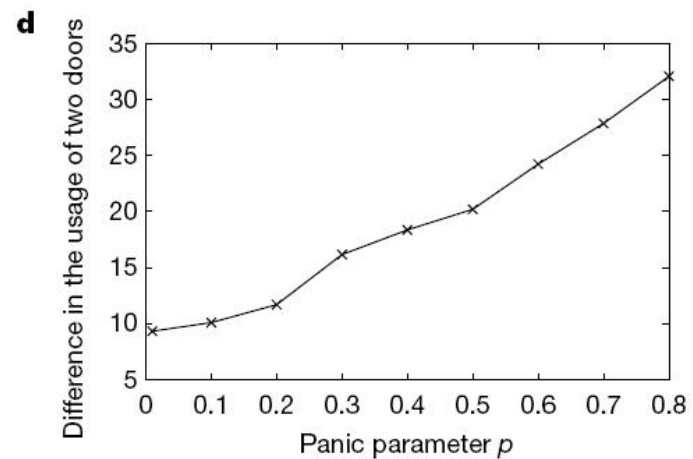
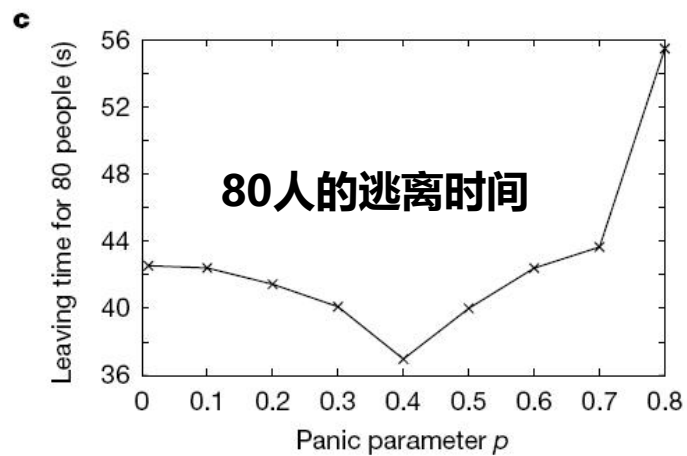
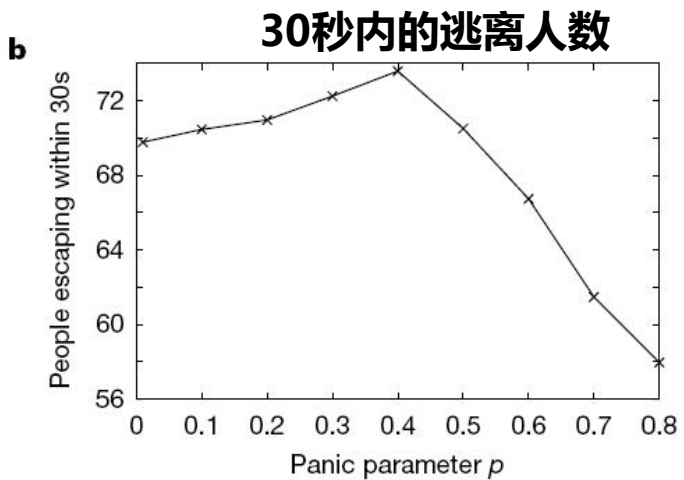
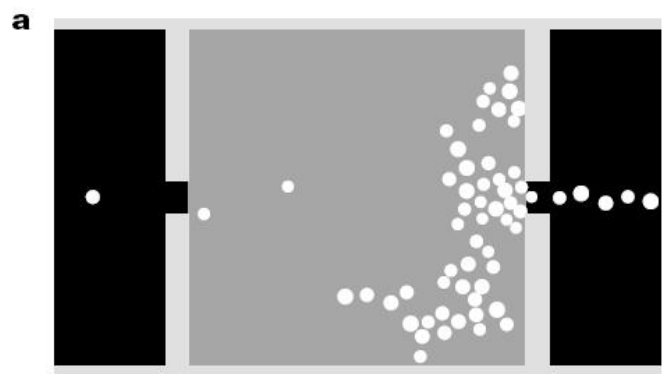


阻塞场景的模拟



“faster-is-slower effect” 欲速则不达效应

羊群效应



若恐慌参数 P 增大，则左右两个出口的人数差异变大

参考源程序

- **C Source code available from** <http://www.panics.org/>

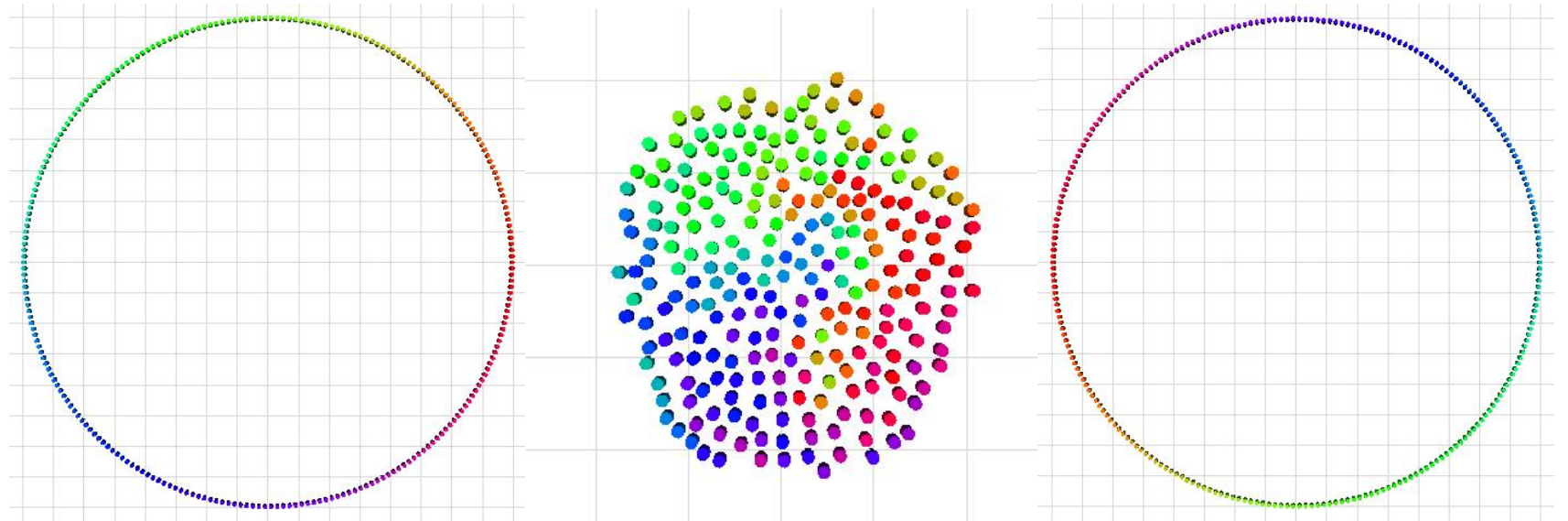
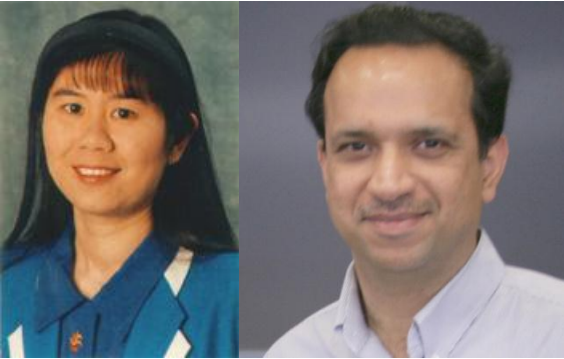


- **Self-Organized Pedestrian Crowd Dynamics-An applet simulating intersecting pedestrian streams**

by Frederik Depta, Lijuan Liu and Nibras A Suleman

<http://itp.uni-frankfurt.de/~gros/JavaApplets/PedestrianCrowdDynamics/PedestrianApplet.html>

相互速度障碍物碰撞避免模型 (RVO, Reciprocal Velocity Obstacles)



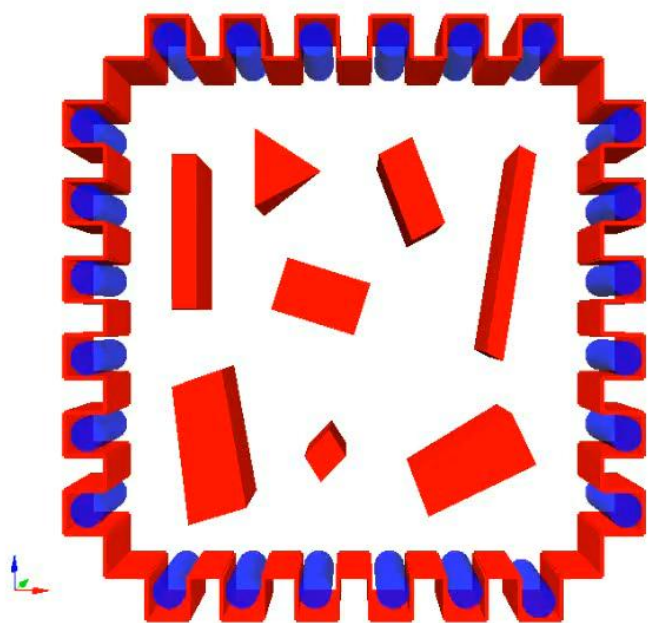
□ 参考文献

Jur van den Berg, Ming Lin, Dinesh Manocha, “**Reciprocal Velocity Obstacles** for Real-Time Multi-Agent Navigation”, IEEE International Conference on Robotics and Automation (ICRA), 2008, pp. 1928-1935.

背景

- 在群组动画中，一个关键问题是如何避免个体之间的碰撞。在现实世界中，当两个行人判断前行可能发生碰撞时，两个人**会同时调整自己的速度**以避免碰撞；
- 尽管碰撞避免是群体动画中常见的行为，但由于涉及到每个个体的运动能力、心理、外界环境等多种因素，这种相互作用复杂性高，合理的碰撞避免是一项极具挑战性的课题；
- **相互速度障碍物(RVO)**模型是一个很好的解决方案
 - 假设个体以**匀速前进**
 - 每个个体在保持与周围个体相对运动的同时，**在速度域中计算出可能导致碰撞的速度集合**，并对自身速度进行必要的调整。在调整过程中，碰撞避免的任务**同时分配给**相关的个体，使它们**相互协调完成**碰撞避免任务。

多智能体(multi-agent)导航



- N 个智能体共享一个环境
- 一个智能体从起始点运动到目标点，相互之间不发生碰撞，并避开障碍物

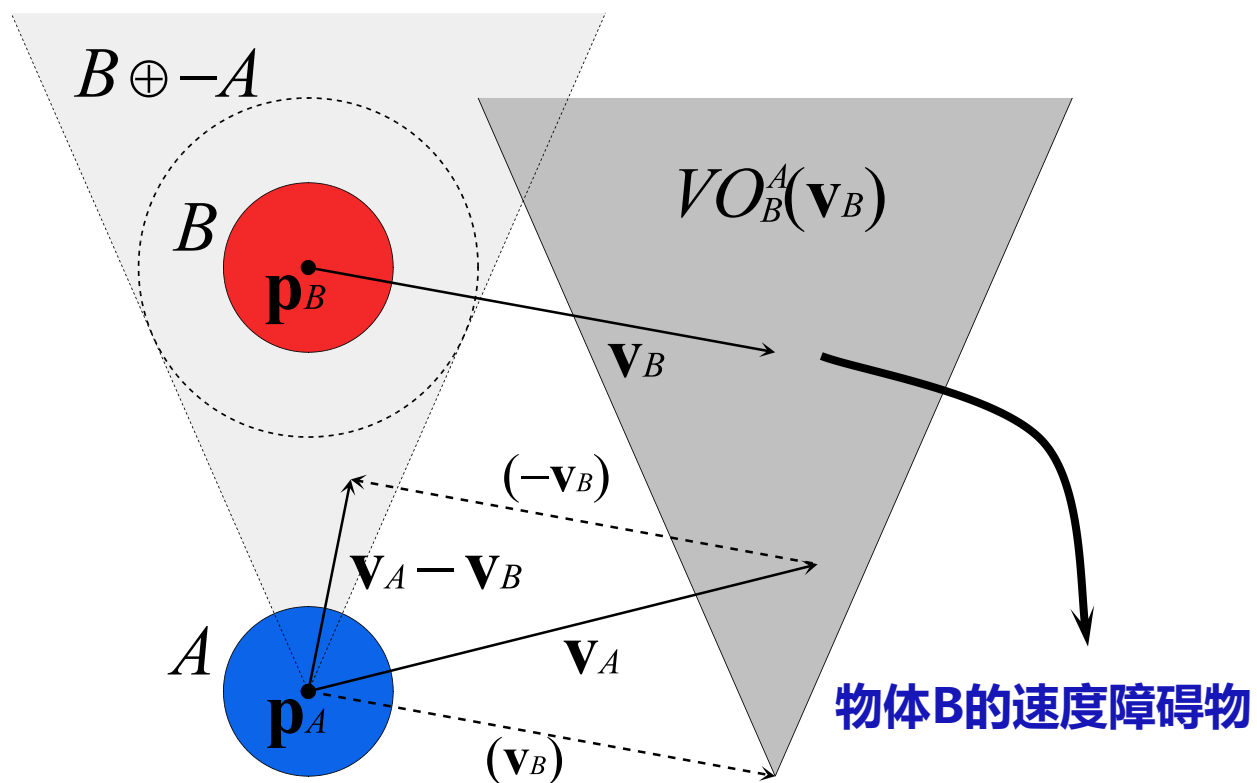
- 解耦处理法(Decoupled)
 - 每个智能体同时自主导航
 - 全局路径规划和局部碰撞避免分开处理
- 集中处理法(Centralized)
 - 在复合配置空间进行路径规划
 - 例子：大型团体操

问题描述

- 独立导航
- 全局路径规划 vs. 局部导航
- 在每个周期：每个智能体能观察到其它智能体(知道其它智能体的位置_{和速度})
- 但是不知道它们准备做什么
- 智能体该如何做？

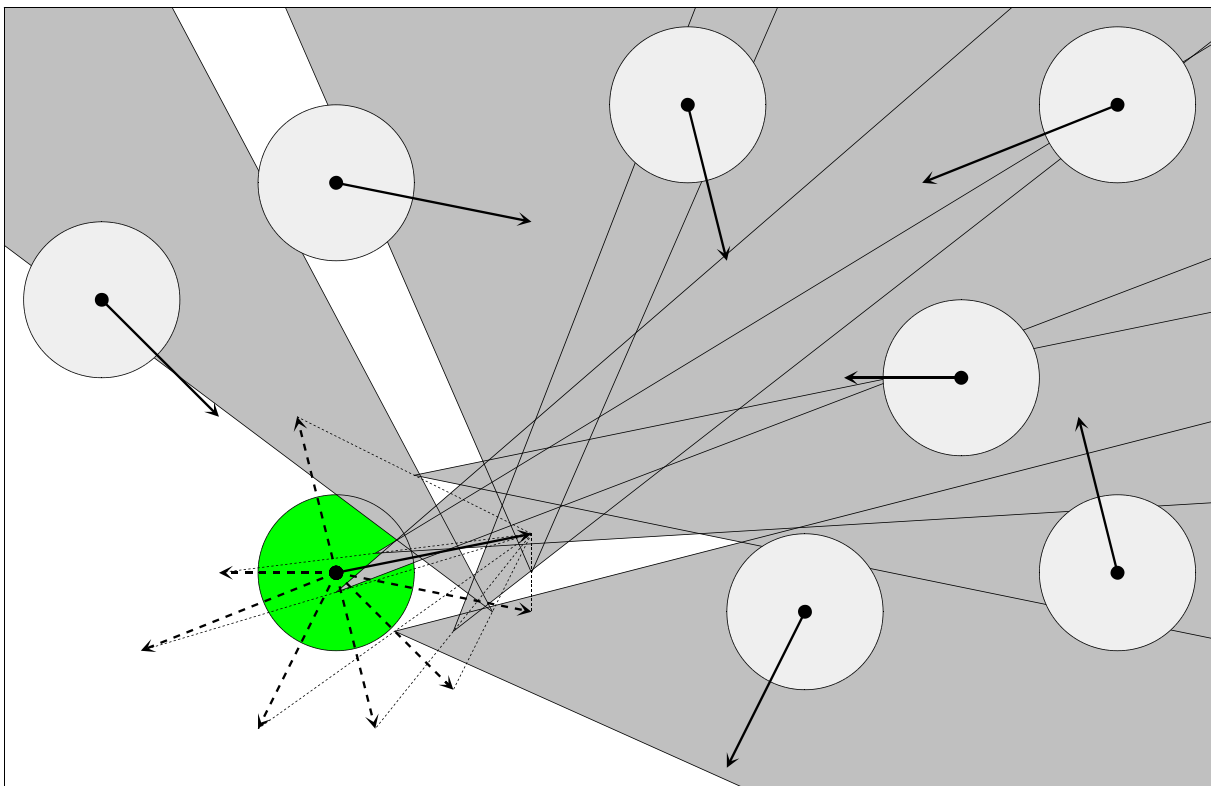
速度障碍物

- 假设 A 为平面上的一个智能体，它的位置 $\mathbf{p}_A$ ， B 为一个移动的平面障碍物，其位置为 $\mathbf{p}_B$
- 则障碍物 B 对智能体 A 的**速度障碍物** $VO_B^A(\mathbf{v}_B)$ 为
 - 在某一时刻将与障碍物 B (以速度 $\mathbf{v}_B$ 移动)碰撞的智能体 A 的所有可能的速度 $\mathbf{v}_A$ 集合
- 在几何上，速度障碍物可以用右面的图来表示：



圆盘形障碍物 B 对一个圆盘形智能体 A 的速度障碍 $VO_B^A(\mathbf{v}_B)$

选择新的速度



- 在相互速度障碍物的并集的外面，
最接近期望的速度

- 环境可能变得很拥挤：没有有效的速度
- 解决方法：在RVO中选择速度，但对下面这些参数进行惩罚：
 - 预计到碰撞的时间
 - 到期期望速度的距离
- 以最小的惩罚选择速度

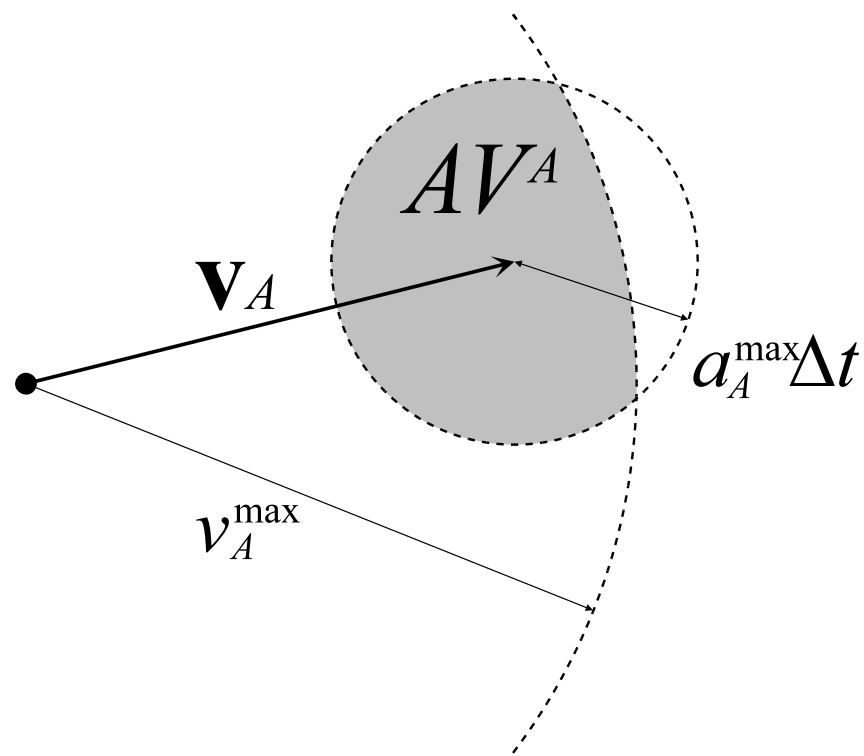
真实性

- 动力学约束

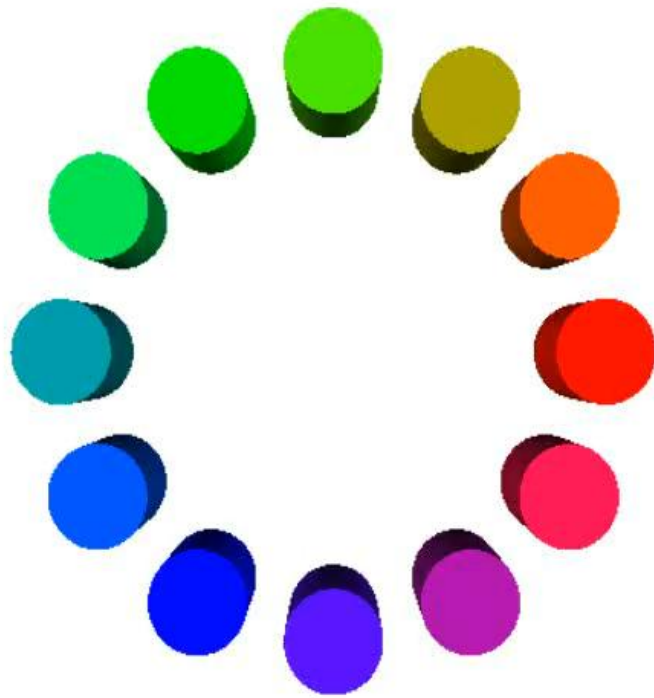
- 最大速度
- 最大加速度
- 其它...

- 邻居

- 圆形邻居
- 可见的邻居区域...



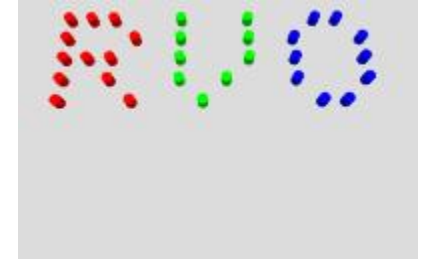
实验



每个智能体运动到圆的正对面

RVO2 Library: Reciprocal Collision Avoidance for Real-Time Multi-Agent Simulation

- 免费下载网页: <http://gamma.cs.unc.edu/RVO2/>
- Latest Versions
 - RVO2 Library and RVO2-3D Library are **open-source software** as of May 4, 2016.
 - RVO2 Library for C++ v2.0.1, released October 26, 2010.
 - RVO2-3D Library for C++ v1.0, released May 13, 2011.
 - RVO2 Library for C# v2.0, released January 10, 2011.
- Introduction
 - Simulations of multiple agents sharing a common workspace have gained increasing attention recently, for various purposes such as crowd simulation, navigating a team of mobile robots, video games, studying natural flocking behavior, traffic engineering, architecture and design, emergency training simulations, etc. We present an algorithm for interactive navigation and planning of large numbers of agents in two-dimensional (crowded) environments. At runtime, each agent senses the environment independently and computes a collision-free motion based on the [optimal reciprocal collision avoidance](#) (ORCA) formalism. Our algorithm ensures that each agent exhibits no **oscillatory** behaviors.
 - *RVO2 Library for C++* is an implementation of our algorithm. It has a simple API for third-party applications. The user specifies static obstacles, agents, and the preferred velocities of the agents. The simulation is performed step-by-step via a simple call to the library. The simulation is fully accessible and manipulable during runtime. The library exploits multiple processors if they are available using [OpenMP](#) for efficient parallelization of the simulation.
 - *RVO2-3D Library for C++* is an implementation of our algorithm for applications in three dimensions without static obstacles. A C# implementation of our algorithm by Slan is available from rvo-unity.chezslan.fr. We are actively developing implementations in C and Java for future release.

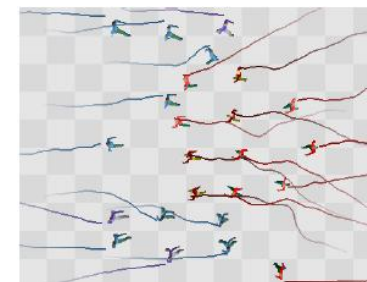


连续人群(Continuum Crowds)

- 目前我们讨论的大都是**基于智能体**的(“agent-based”)方法。
- 对每个个体，独立计算其运动。
- 这类方法能刻画每个人的独特情况：
 - Visibility
 - Proximity of other pedestrians
 - Other local factors
- 每个个体可定义不同的模拟参数。

缺点：

- 难以一致地生成连续的运动
- 对每个个体进行全局的路径规划代价较大：
 - 大部分模型把局部碰撞检测与全局的路径规划分开处理。
 - 会导致冲突（全局vs局部）。



连续人群(Continuum Crowds)



- Treuille等人提出了基于连续体动力学的实时人群模型。在该模型中，**动态势能场**同时将全局导航与移动障碍物（例如其他人）整合在一起，**无需明确碰撞避免**即可有效地解决大规模人群的运动。仿真可以以**交互速率**运行，在各种条件下平稳流动，并自然展现了在真实人群中观察到的紧急现象。
- Treuille, A., Cooper, S. and Popović, Z., 2006. Continuum crowds. ACM Transactions on Graphics (TOG), 25(3), pp.1160-1168. (Google Citations: **1325**, 2026.4)

方法流程

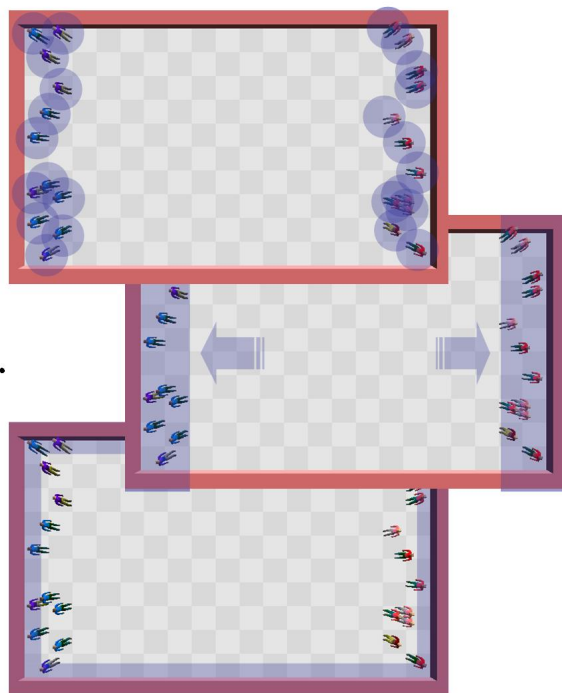
Compute a set of grids representing state.

Density Grid: Indicates people's static locations.

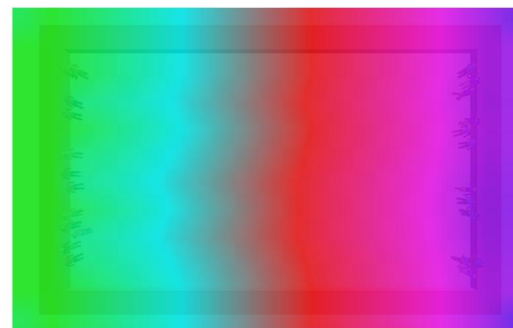
Goal Grids: Indicate people's desired locations.

Boundary Grids: Indicate impassible grid cells.

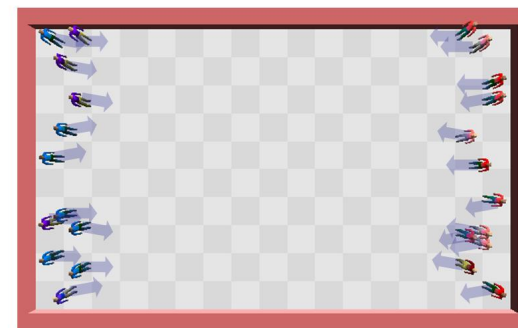
Other grids...



Combine grids into a set of potential fields.



Use potential fields to update people's positions.



控制方程(Governing Equations)

- **假设1:** 每个人尽力到达一个地理目标 $G \subseteq \mathbb{R}^2$
- **假设2:** 人群尽可能按最大的速度运动。这可表达为一个最大速度场 f , 使得在 $\mathbf{x}$ 位置的一个人在沿 θ 方向运动时, 其速度为: $\dot{\mathbf{x}} = f(\mathbf{x}, \theta) \mathbf{n}_\theta$ 。其中, $\mathbf{n}_\theta = (\cos\theta, \sin\theta)^T$ 表示指向 θ 方向的单位矢量。
- **假设3:** 存在一个不适场 g , 如果 $g(\mathbf{x}') > g(\mathbf{x})$, 则人们更倾向在 $\mathbf{x}$ 而非 $\mathbf{x}'$ 。

控制方程(Governing Equations)

- **假设4:** 假设 Π 为从 $\mathbf{x}$ 到目标某个点的所有路径的集合。假设速度场 f 、不适场 g 、目标 G 固定, 在位置 $\mathbf{x}$ 的人将选择路径 $P \in \Pi$, 并使得下面的能量极小化:

$$\underbrace{\alpha \int_P 1 ds}_{\text{Path Length}} + \underbrace{\beta \int_P 1 dt}_{\text{Time}} + \underbrace{\gamma \int_P g dt}_{\text{Discomfort}} .$$

其中 α 、 β 、 γ 为权因子, ds 表示沿弧长积分, dt 表示沿时间积分。因为 $ds=f dt$, 故

$$\alpha \int_P 1 ds + \beta \int_P \frac{1}{f} ds + \gamma \int_P \frac{g}{f} ds \implies \int_P C ds, \quad \text{where} \quad C \equiv \frac{\alpha f + \beta + \gamma g}{f}$$

单位代价场(Unit cost field)

最优路径计算

(Optimal Path Computation)

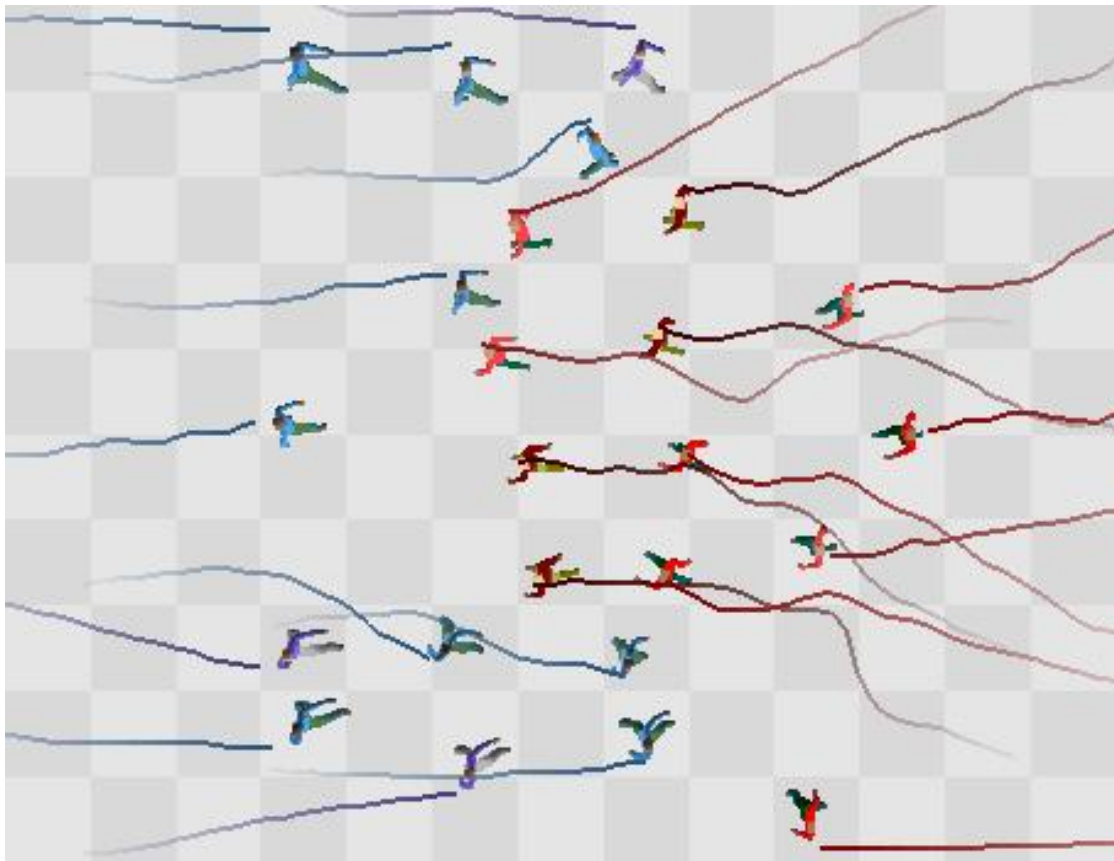
- 给定单位代价场(Unit cost field), 采用势能场方程来求解最优路径
- 假设我们有一个**动态梯度函数** $\phi: \mathbf{R}^2 \rightarrow \mathbf{R}$, 其值等于到达目标最佳路径的代价。
- 直观地, 对于每个人, 最优策略是沿着这个函数梯度相反的方向运动, 因为这样可以最快地降低路径的代价。
- **动态梯度函数**可采用如下方法来定义:
 - 在目标处: $\phi=0$,
 - 在其它地方满足Eikonal方程, $\|\nabla \phi(\mathbf{x})\| = C$,
- 这表示, 每个人沿着梯度相反的方向运动, 其速度为 $\dot{\mathbf{x}} = f(\mathbf{x}, \theta) \mathbf{n}_\theta$, 也即:

$$\dot{\mathbf{x}} = -f(\mathbf{x}, \theta) \frac{\nabla \phi(\mathbf{x})}{\|\nabla \phi(\mathbf{x})\|},$$

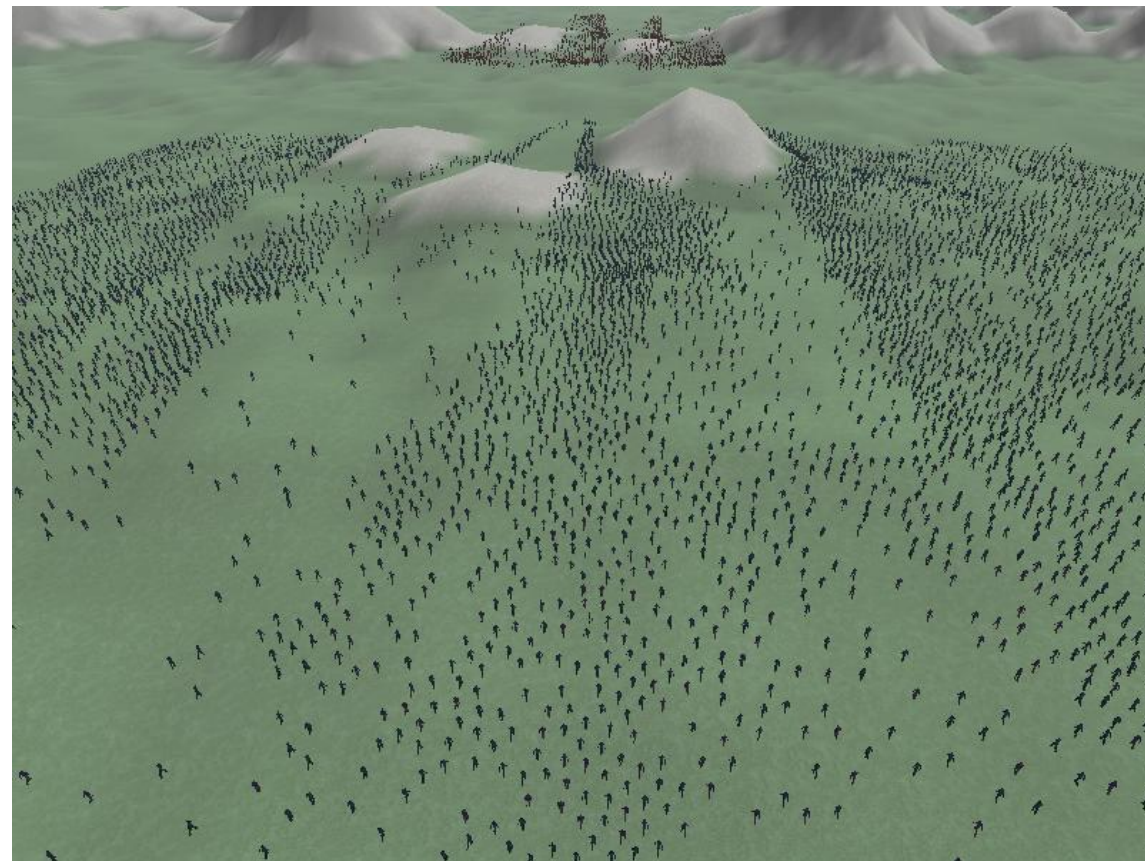
算法流程

- 对于每个时间步:
 - 把crowd转化为密度场;
 - 对于每个group:
 - 构造单位代价场 C ;
 - 构造势能场 ϕ 及其梯度 $\nabla \phi$
 - 更新每个人的位置
 - 强制保证人之间的最短距离

结果



连续人群(蓝色)与Reynold智能体人群互动(红色)



一支大部队实时穿过山区地形去追逐一支较小的部队

DEMO



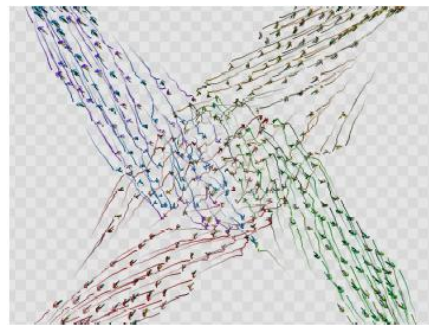
Continuum Crowds

Adrien Treuille
Seth Cooper
Zoran Popović

优缺点

优点

- 可消除个体的运动冲突现象
- 运动路径更光滑
- 可方便地整合到基于智能体的方法中
- 可刻画一些常见的人群现象，如
 - 人行道的形成
 - 堵塞期间的短时涡旋



缺点

- 不适用于未知的环境
 - 该方法假设人群知道动态的真实环境
- 在不考虑惯性的情况下改变方向
- 与完全基于Agent的方法相比，缺少灵活性和个体差异性

The End